

Model Checking

- Logiche temporali
- Automi e logiche temporali
- Modelli per sistemi concorrenti:
 - Process Algebra
 - Reti di Petri

Contesto Statico: Queste logiche operano su un singolo, immutabile stato o mondo. Una formula phi viene valutata rispetto a questo singolo stato di cose.
- Valore di Verità Fissato: Un simbolo proposizionale, come "è lunedì", una volta assegnato un valore di verità in quel "mondo" (per esempio, il 30 Novembre 2025), mantiene quel valore per tutta la valutazione. Non c'è un meccanismo intrinseco per modellare il cambiamento nel tempo.

Le logiche di proposizione e di predicato sono la base, ma non bastano

Logiche temporali



- Logiche classiche considerano un unico "mondo" sul quale esprimono proprietà che sono sempre vere o false
 - Esempio: la proposizione "è lunedì" risulta essere o sempre vera o sempre falsa
- Ma i sistemi software sono sistemi "dinamici"
 - Esempio: la proposizione "la variabile x contiene il valore 10" puo' essere a volte vera a volte falsa
- Le logiche temporali permettono di esprimere proprietà temporali:
 - vere in alcuni "mondi", false in altri

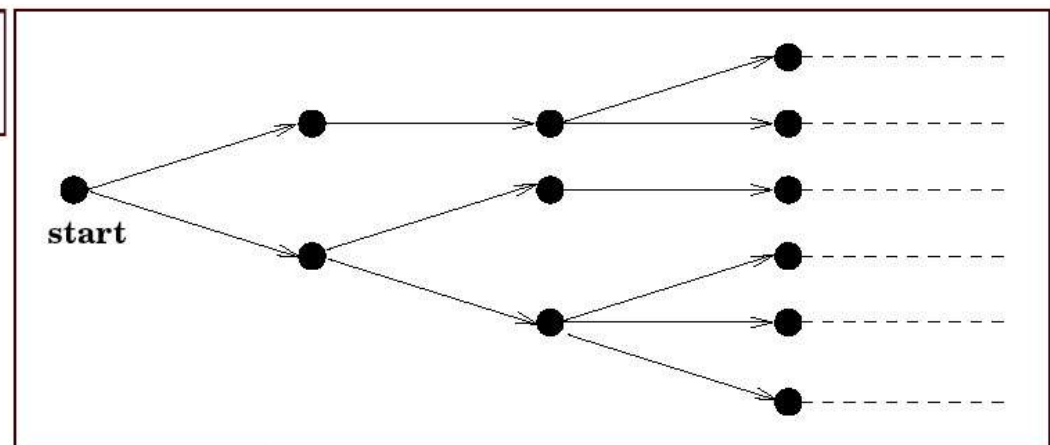
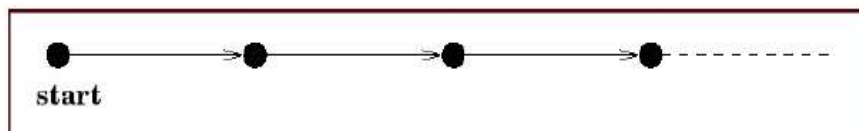
Ho bisogno di aggiungere alle logiche standard degli aspetti dinamici

Le Logiche Temporali sono un'estensione delle logiche classiche che aggiungono degli operatori temporali per esprimere come la verità delle proposizioni cambia nel tempo lungo una sequenza di "mondi successivi".

Il tempo é rappresentato in modo discreto (a step)

Logiche temporali

- I “mondi” considerati corrispondono a diversi momenti temporali
- Possono esistere diversi modi per mettere in relazione questi “mondi”
 - Linear time:
 - Branching time



Sequenze lineari di "mondi"

- Una sequenza linear time di "mondi" è quindi una sequenza di insiemi di proposizioni che indicano i fatti veri in quell'istante temporale
- Esempi: In ogni istante, ho proposizione vere in quel "mondo", mentre le altre sono false

● → ● → ● → ● → ● → ● ...
{lun} {mar} {mer} {gio} {ven} {sab}

● → ● → ● → ● → ● → ● ...
{s} {s,v} {p} {} {v} {s}
s=sole v=vento p=pioggia

LTL estende la logica proposizionale introducendo gli operatori temporali che ci permettono di ragionare sulle proprietà lungo una singola sequenza lineare di "mondi" successivi del sistema.

Linear Temporal Logic (LTL)

- Estende la logica proposizionale con operatori per specificare proprietà su sequenze linear time di "mondi":

$\bigcirc \varphi$	φ is true in the <i>next</i> moment in time	Vero nel prossimo "mondo"
$\Box \varphi$	φ is true in <i>all</i> future moments	Vero da ora in poi all'infinito
$\Diamond \varphi$	φ is true in <i>some</i> future moment	Vero in qualche "mondo" futuro
$\varphi \mathcal{U} \psi$	φ is true <i>until</i> ψ is true	psi diventerà vera, e fino ad allora phi è vera.

- Esempio:

$$\Box((\neg passport \vee \neg ticket) \Rightarrow \bigcirc \neg board_flight)$$

Altri esempi

- Sistema in cui si richiede l'esecuzione di un task:
tale richiesta viene ricevuta, elaborata, ed eseguita

$$\Box(requested \Rightarrow \Diamond received)$$

$$\Box(received \Rightarrow \bigcirc processed)$$

$$\Box(processed \Rightarrow \Diamond \Box done)$$

Ci sarà un istante i da cui
sarà per sempre completato

- In tale sistema, se si richiede l'esecuzione del task,
questo sarà sicuramente eseguito. Formalmente la
seguente proprietà, per un tale sistema, è falsa:

$$requested \wedge \Box \neg done$$

Sintassi LTL

$\varphi, \psi \rightarrow$	$p \mid$	(atomic proposition)
	$\top \mid$	(true)
	$\perp \mid$	(false)
	$\neg\varphi \mid$	(complement)
	$\varphi \wedge \psi \mid$	(conjunction)
	$\varphi \vee \psi \mid$	(disjunction)
	$\bigcirc\varphi \mid$	(next time)
	$\Box\varphi \mid$	(always)
	$\Diamond\varphi \mid$	(sometime)
	$\varphi \text{ } \mathcal{U} \text{ } \psi$	(until)

Modelli: sequenze di mondi

Le formule della LTL sono costruite a partire da un insieme di proposizioni atomiche (P) (spesso indicate come $(p, q, r, \dots)$) utilizzando operatori logici classici e operatori temporali.

- Si considerino **formule di LTL** che utilizzano **proposizioni $p \in P$**

Il Model Checking parte da un insieme di simboli proposizionali P . Queste proposizioni sono gli eventi o fatti base del sistema che possono essere veri o falsi.

- Un **modello** π è una **sequenza infinita** $\pi_1, \pi_2, \pi_3, \dots$ di "mondi" dove π_i è una funzione:

$$\pi_i : P \rightarrow \{T, F\}$$

funzione di valutazione che assegna un valore di verità (Vero o Falso) a tutte le proposizioni atomiche in quell'istante temporale

- **Nota:** nei disegni visti si rappresenta l'insieme delle proposizioni vere

Nei diagrammi e nei disegni che rappresentano gli stati, spesso non si disegna l'intera funzione (quindi anche le prop. false), ma si elencano solo le proposizioni che sono Vere in quello stato.

Semantica di LTL

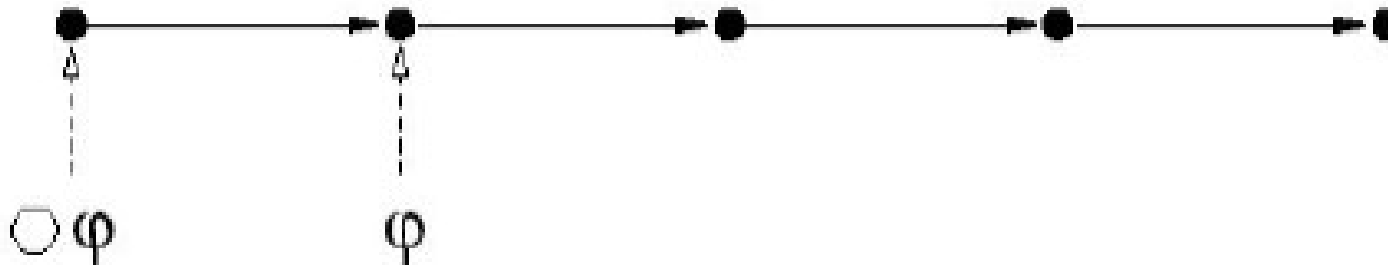
Proposizione Atomica (p)
- Spiegazione: Una proposizione atomica p è soddisfatta da un modello π se e solo se p è vera nel primo "mondo" di quel modello (π_1).
- Nella logica proposizionale classica, la formula è valutata in un singolo "mondo". Qui, sebbene si valuti solo un singolo "mondo" (π_1), stiamo localizzando la verità all'inizio di una sequenza temporale infinita, che è la novità.

Definizione della relazione $\pi \models \phi$

- $\pi \models p \iff \pi_1(p) = T$ (unico diverso da logica proposizionale)
- $\pi \models \top, \pi \not\models \perp$
- $\pi \models \neg\phi \iff \text{not } \pi \models \phi$
- $\pi \models \phi \wedge \phi' \iff \pi \models \phi \text{ and } \pi \models \phi'$
- $\pi \models \phi \vee \phi' \iff \pi \models \phi \text{ or } \pi \models \phi'$
- $\pi \models \phi \rightarrow \phi' \iff \pi \models \phi \text{ implies } \pi \models \phi'$
- ...

Operatore "next"

- Usato per indicare proposizioni vere nel prossimo momento temporale:



Esempi:

$$(sad \wedge \neg rich) \Rightarrow \bigcirc sad$$

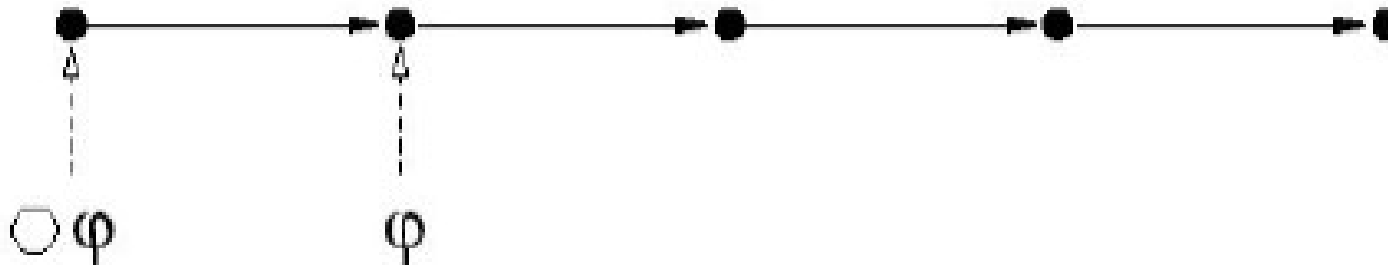
$$((x = 0) \wedge add3) \Rightarrow \bigcirc (x = 3)$$

①

Operatore "next"

L'operatore Next è utilizzato per fare un'affermazione di verità strettamente sul passo temporale immediatamente successivo.

- Usato per indicare proposizioni vere nel prossimo momento temporale:



Semantica:

Sia $\pi = \pi_1, \pi_2, \pi_3, \dots$ il cammino di esecuzione.

Sia π^i la sequenza infinita ottenuta da π a partire dal mondo (stato) i -esimo: $\pi^i = \pi_i, \pi_{i+1}, \pi_{i+2}, \dots$

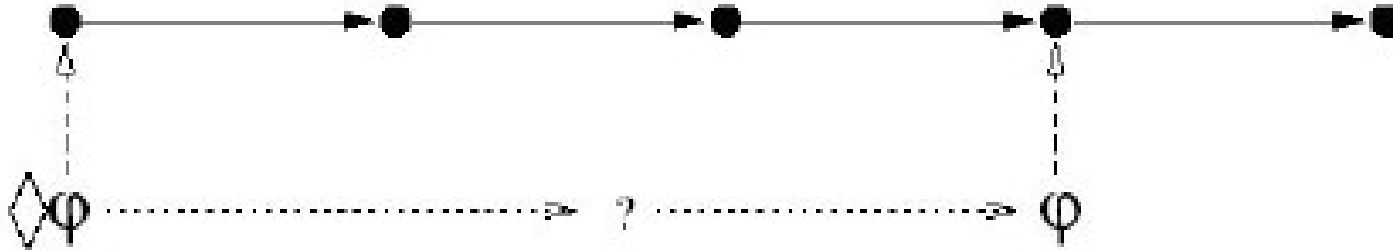
sequenza infinita di mondi che parte dal secondo mondo della sequenza di mondi π

$$\pi \models \bigcirc \varphi \iff \pi^2 \models \varphi$$

Con π^i si rappresenta la sequenza infinita ottenuta da π partendo dal mondo i -esimo

Operatore "sometime"

- Usato per indicare proposizioni vere ora o in un qualche momento futuro:



Esempi:

$$(\neg \text{resigned} \wedge \text{sad}) \Rightarrow \Diamond \text{famous}$$

$$\text{sad} \Rightarrow \Diamond \text{happy}$$

$$\text{send} \Rightarrow \Diamond \text{receive}$$

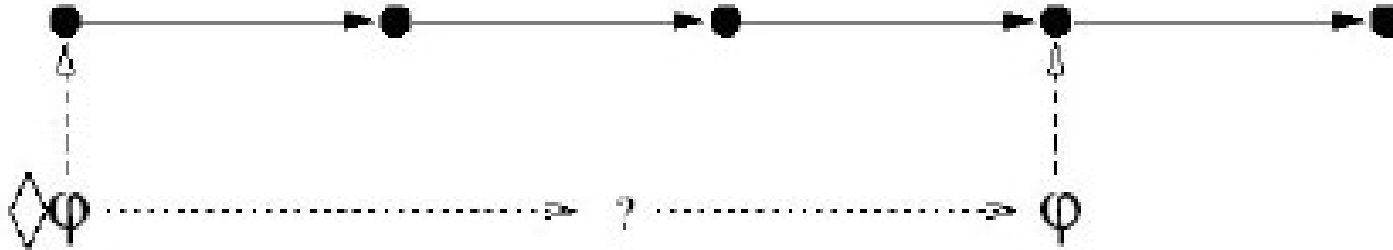
②

Operatore "sometime"



L'operatore Sometime è usato per specificare che una proposizione phi deve diventare vera in un punto del futuro.

- Usato per indicare proposizioni vere ora o in un qualche momento futuro:

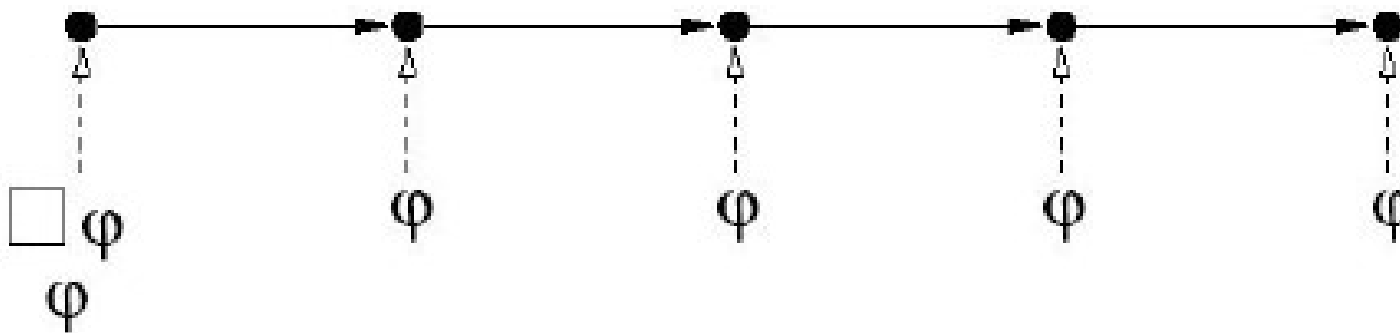


Semantica:

$$\pi \models \Diamond \varphi \iff \text{there is some } i \geq 1 \text{ such that } \pi^i \models \varphi$$

Operatore "always"

- Usato per indicare proposizioni vere ora e in tutti i momenti futuri:



Esempi:

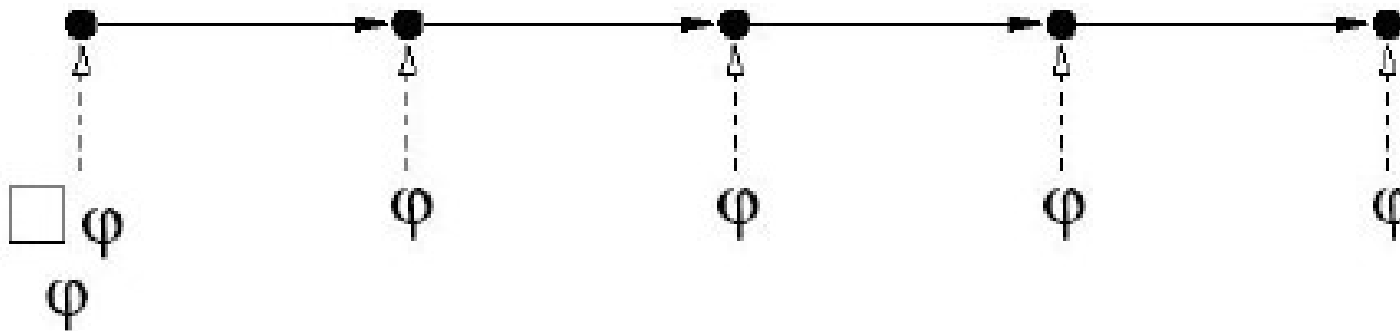
lottery-win $\Rightarrow$ $\square$ *rich*

3

Operatore "always"

- L'operatore Always è usato per specificare che una proposizione ϕ deve essere continuamente vera per tutta l'esecuzione futura del sistema, incluso lo stato corrente.

Usato per indicare proposizioni vere ora e in tutti i momenti futuri:

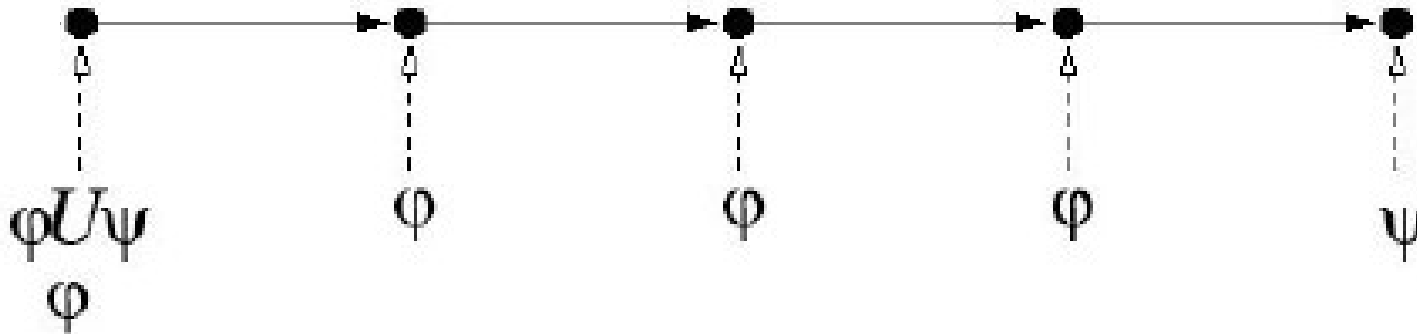


Semantica:

$$\pi \models \Box \phi \iff \text{for all } i \geq 1 \text{ we have } \pi^i \models \phi$$

Operatore “until”

- Usato per mettere in relazione due proposizioni, una vera ora ed in tutti momenti successivi fino ad un momento in cui è vera la seconda:



Esempi:

start_lecture $\Rightarrow$ *talk* $\cup$ *end_lecture*

born $\Rightarrow$ *alive* $\cup$ *dead*

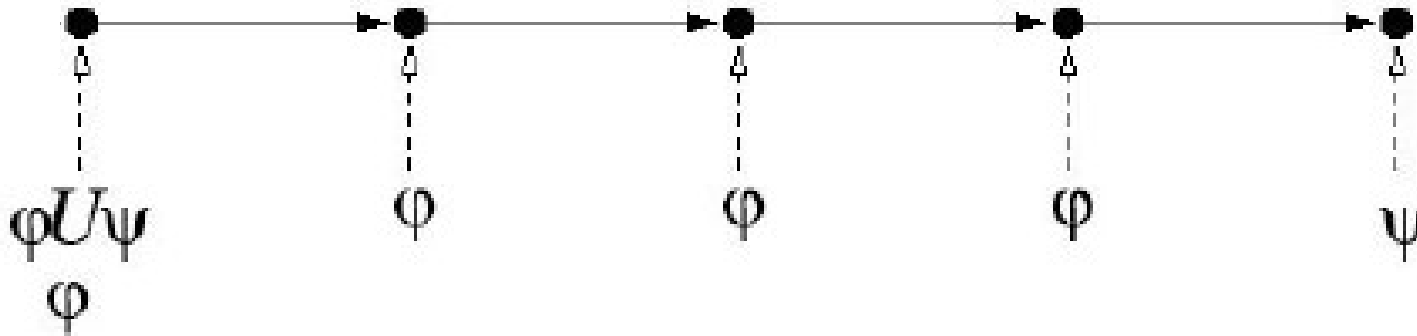
request $\Rightarrow$ *reply* $\cup$ *acknowledgement*

4

Operatore "until"

L'operatore Until è l'operatore LTL più potente perché definisce una relazione di sequenza stretta e limitata tra due proposizioni, phi e psi:
Viene utilizzato per specificare scenari in cui una condizione deve persistere fino al verificarsi di un'altra condizione futura, che è garantita.

- Usato per mettere in relazione due proposizioni, una vera ora ed in tutti momenti successivi fino ad un momento in cui è vera la seconda:



Semantica:

psi deve diventare vera prima o poi in un qualche punto del futuro. Questo garantisce che il futuro non sia infinito senza che psi accada.
phi deve essere vera in tutti gli stati precedenti al momento in cui psi diventa vera.

$$\pi \models \varphi u \psi \iff \text{there is some } i \geq 1 \text{ such that } \pi^i \models \psi \text{ and} \\ \text{for all } j = 1 \dots i-1 \text{ we have } \pi^j \models \varphi$$

Operatori minimali

■ In logica, dire che due operatori sono duali significa che puoi esprimere l'uno nei termini dell'altro semplicemente "circondandolo" di negazioni (not).

■ "always" e "sometime" sono operatori duali:

$$\neg \Box \varphi \equiv \Diamond \neg \varphi$$

("Non è sempre vero phi"): Significa che esiste almeno un momento, ora o in futuro, in cui phi smette di essere vera.

("Prima o poi sarà vero not phi"): Significa esattamente la stessa cosa: nel flusso del tempo, incontreremo uno stato che soddisfa not phi.

■ "sometime" può essere espresso usando "until":

$$\Diamond \varphi \equiv \top \mathcal{U} \varphi$$

■ Di conseguenza tutti gli operatori possono essere espressi usando solo "next" e "until"

Relazione con operatori classici

Gli operatori NON distribuiscono in modo incrociato: cioè il sometime non distribuisce con l'AND e always non distribuisce con l'OR

- "sometime" distribuisce su or, "always" su and:

$$\Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$$

Se è vero che prima o poi accadrà phi oppure psi, è equivalente a dire che prima o poi accadrà phi OPPURE prima o poi accadrà psi.

$$\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$$

Se è vero che sempre accadrà phi AND psi, è equivalente a dire che sempre accadrà phi AND sempre accadrà psi.

- Relazione con il not:

L'operatore next si comporta come un'operazione che non cambia la semantica della negazione, permettendole di "scivolare" all'interno della formula.

$$\neg \bigcirc \varphi \equiv \bigcirc \neg \varphi$$

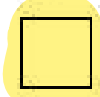
Non è vero che nel prossimo momento phi è vera, è equivalente a dire che nel prossimo momento phi è falsa.

$$\neg(\varphi \mathcal{U} \psi) \equiv (\neg\psi \mathcal{U} (\neg\varphi \wedge \neg\psi)) \vee \Box\neg\psi$$

cioè φ diviene falso prima che ψ sia vero oppure ψ non è mai vero

Notazione alternativa

- Notazione standard testuale (usata da nostro libro) per gli operatori temporali: "U" per "until" e

	$\rightsquigarrow$	<i>F</i>	sometime in the Future
	$\rightsquigarrow$	<i>G</i>	Globally in the future
	$\rightsquigarrow$	<i>X</i>	neXtime

Oppure semplicemente (usata dal nostro tool):

<>	sometime
[]	always
()	next

Proprietà tipiche per sistemi software

- Esistono utili concetti che tipicamente sono importanti nei sistemi informatici:

- Proprietà di "**safety**":
non si raggiungono mai stati con errori

Le proprietà di Safety affermano che "niente di cattivo accadrà mai". Garantiscono che il sistema rimanga all'interno di un'area operativa sicura.

- Concetto Chiave: Non si raggiungono mai stati con errori.
- Violazione: Una proprietà di Safety è violata se si verifica un evento indesiderato (il "cattivo" evento) anche solo una volta. La violazione è sempre visibile su un prefisso finito del cammino (cioè, un Model Checker può mostrare lo stato di errore dopo un numero finito di passi).

- Proprietà di "**liveness**":
prima o poi si eseguirà una certa azione

Le proprietà di Liveness affermano che "qualcosa di buono accadrà prima o poi". Garantiscono che il sistema progredisca e non si fermi indefinitamente.

- Concetto Chiave: Prima o poi si eseguirà una certa azione.
- Violazione: Una proprietà di Liveness è violata se l'evento atteso non accade mai. La violazione non è visibile su un prefisso finito, ma solo su un cammino infinito che non contiene mai l'evento.

- Proprietà di "**fairness**":
se si richiede una cosa infinite volte, questa verrà eseguita infinite volte

All'infinito, una richiesta deve essere soddisfatta

Le proprietà di Fairness sono un caso speciale di Liveness e sono necessarie per la verifica di sistemi concorrenti e distribuiti. Definiscono condizioni sul non determinismo del sistema, assumendo che i componenti non si comportino in modo costantemente arbitrario o malizioso.

- Concetto Chiave: Se si richiede una cosa infinite volte, questa verrà eseguita infinite volte (o, in generale, se un evento è continuamente abilitato, prima o poi deve essere scelto).

Safety

- “qualcosa di cattivo non accadrà mai”

$$\square \neg (reactor_temp > 1000)$$

- Proprietà di “safety” si esprimono tipicamente così:

$$\square \neg \dots$$

Liveness

- “qualcosa di buono accadrà”

$\diamond rich$

$\diamond (x > 5)$

$\square (start \Rightarrow \diamond terminate)$

$\square (Trying \Rightarrow \diamond Critical)$

- Proprietà di “liveness” si esprimono solitamente così:

$\diamond \dots$

Fairness

- "se qualcosa è richiesta infinite volte, allora avverrà infinite volte"

$$\square \diamond ready \Rightarrow \square \diamond run$$

"Se il processo è pronto infinitamente spesso, allora deve essere eseguito infinitamente spesso."

Perché uso LTL? LTL è un frammento decidibile della logica dei predicati (esiste un algo che termina sempre nella valutazione di soddisfacibilità)

può essere formalmente vista

LTL come frammento decidibile della logica dei predicati

Una formula LTL ϕ viene codificata come una formula della Logica del Primo Ordine, $\llbracket \phi \rrbracket_z$, dove $\llbracket \phi \rrbracket_t$ è la sua traduzione in un'espressione che dipende dalla variabile temporale t . La traduzione è definita per induzione sulla struttura della formula ϕ .

- Tempo Discreto: Il tempo è modellato dalla variabile t , che assume valori in $\mathbb{N}$.
 - Stato Iniziale: Lo stato iniziale (π_1) è codificato dal numero 0, rappresentato dalla costante z .
 - Transizione: La transizione al prossimo stato (π_{i+1}) è modellata dalla funzione successore $s(t)$. $s(0)=1$, $s(1)=2$, e così via.
 - Proposizioni Atomiche: Una proposizione atomica p è vera nello stato π_i (dove $i \geq 1$) se e solo se il predicato unario $p(i-1)$ è vero.
 $p(t)$ significa "p è vera al tempo t".

- Le proposizioni $\pi_i(p)$ per $i \geq 1$ sono codificate tramite predicati unari $p(i)$ su numeri naturali
 - costante z associata a 0 e $s(i)$ funzione successore

Una formula LTL ϕ è codificata dalla formula $\llbracket \phi \rrbracket_{s(z)}$, dove:

t, t', t'' sono variabili che rappresentano istanti temporali (numeri naturali)

$$\llbracket p \rrbracket_t = p(t) \quad \text{La proposizione } p \text{ è vera all'istante } t.$$

$$\llbracket \bigcirc \phi \rrbracket_t = \llbracket \phi \rrbracket_{s(t)} \quad \text{phi è vera nel prossimo istante, che è dato dalla funzione successore di } t.$$

$$\llbracket \Diamond \phi \rrbracket_t = \exists t' \ t' \geq t \wedge \llbracket \phi \rrbracket_{t'} \quad \text{phi è vera prima o poi (Finally): esiste un istante } t' \text{ successivo o uguale a } t \text{ in cui phi è vera. (Il tempo } t' \text{ è quantificato).}$$

$$\llbracket \Box \phi \rrbracket_t = \forall t' \ t' \geq t \rightarrow \llbracket \phi \rrbracket_{t'} \quad \text{phi è vera sempre (Globally): per ogni istante } t' \text{ successivo o uguale a } t, \text{ phi è vera. (Il tempo } t' \text{ è quantificato universalmente).}$$

$$\llbracket \phi \cup \psi \rrbracket_t = \exists t' \ t' \geq t \wedge \llbracket \psi \rrbracket_{t'} \wedge \forall t'' \ (t \leq t'' \wedge s(t'') \leq t') \rightarrow \llbracket \phi \rrbracket_{t''}$$

ψ è vera finalmente (t'), e per tutti gli istanti intermedi (t'') strettamente prima di t' , ϕ è vera.

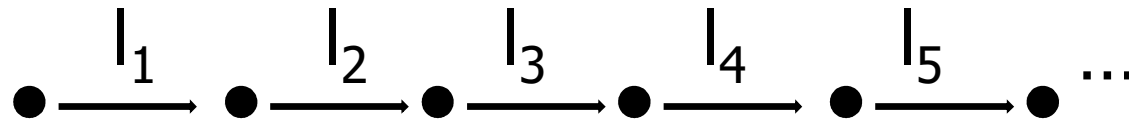
Labeled Transition Systems e Logiche Temporal

Mi interessa il
comportamento del
sistema, non mi interessa
accettare

- Un **Labeled Transition System (LTS)** è un **NFA senza** stati di accettazione **F**

- è una quadrupla (Q, Σ, δ, q_0) con Σ insieme di etichette

- Una **traccia** di un LTS è una sequenza di etichette



- Le tracce ricordano le stringhe degli NFA: iniziano dallo stato iniziale, ma ci interessano le **massimali**
 - lunghezza infinita o terminano in uno stato di **blocco** (no transizioni in uscita)

Formalmente...

- Una **traccia** σ di un **LTS** (Q, Σ, δ, q_0) è una sequenza di etichette $l_1 l_2 \dots$ tale che esistono stati $q_1, q_2, \dots$ con $q_i \in \delta(q_{i-1}, l_i)$ per ogni $i = 1 \dots \text{len}(\sigma)$
 - $\text{len}(\sigma)$ è la lunghezza della sequenza (può valere ∞)
- Una **traccia** σ è **massimale** se $\text{len}(\sigma) = \infty$ oppure se $q_{\text{len}(\sigma)}$ è di blocco, cioè $\delta(q_{\text{len}(\sigma)}, l) = \emptyset$ per ogni $l \in \Sigma$

sigma minuscolo

q_i appartiene ad un cammino del NFA

Non ci sono cammini raggiungibili da $q_{\text{len}(\sigma)}$

Un Labeled Transition System (LTS) è un formalismo molto comune per modellare il comportamento dei sistemi concorrenti.

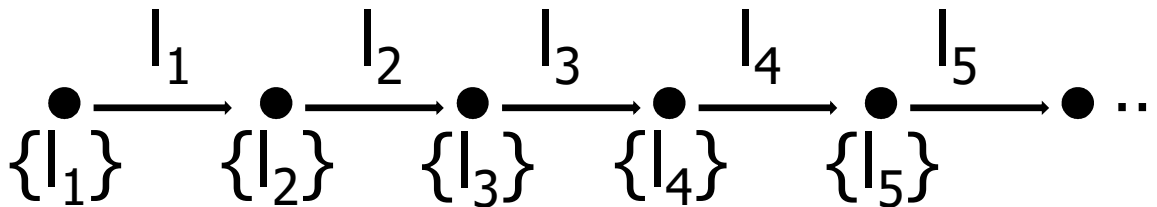
- Stati: Nodi che rappresentano gli stati possibili del sistema.
- Transizioni (Archi): Frece tra gli stati che sono etichettate con le azioni o gli eventi che causano il cambiamento di stato.

Esprimere proprietà di LTS con formule di logica temporale



Convenzione (Trace-based Semantics): La proposizione atomica vera in un dato stato s di una traccia (il "mondo" π_i) è l'etichetta dell'unica transizione uscente da quello stato in quella traccia. Se, da uno stato s , la traccia scelta è $s \xrightarrow{I} s'$, allora l'interpretazione dello stato s è che la proposizione I è vera, e tutte le altre etichette di transizione sono false. Questo ci permette di esprimere le formule LTL direttamente in termini di azioni (es. "L'azione richiesta sarà seguita dall'azione risposta").

In ogni stato ("mondo") attraversato lungo una **traccia** vale **una sola proposizione** coincidente con l'**etichetta** della transizione successiva:



Significa che la verifica di una proprietà LTL su un sistema modellato come un LTS deve essere universale e assoluta su tutti i possibili comportamenti del sistema.

Un LTS soddisfa una **formula** di LTL se **tutte le sue tracce massimali** soddisfano tale formula

Formalmente...

- Una **traccia** $\sigma = l_1 l_2 \dots$ di un LTS (Q, Σ, δ, q_0) **individua il modello** π (per formule LTL con proposizioni su Σ) definito da:

$$\pi_i(p) = T \iff i \leq \text{len}(\sigma) \wedge p = l_i$$

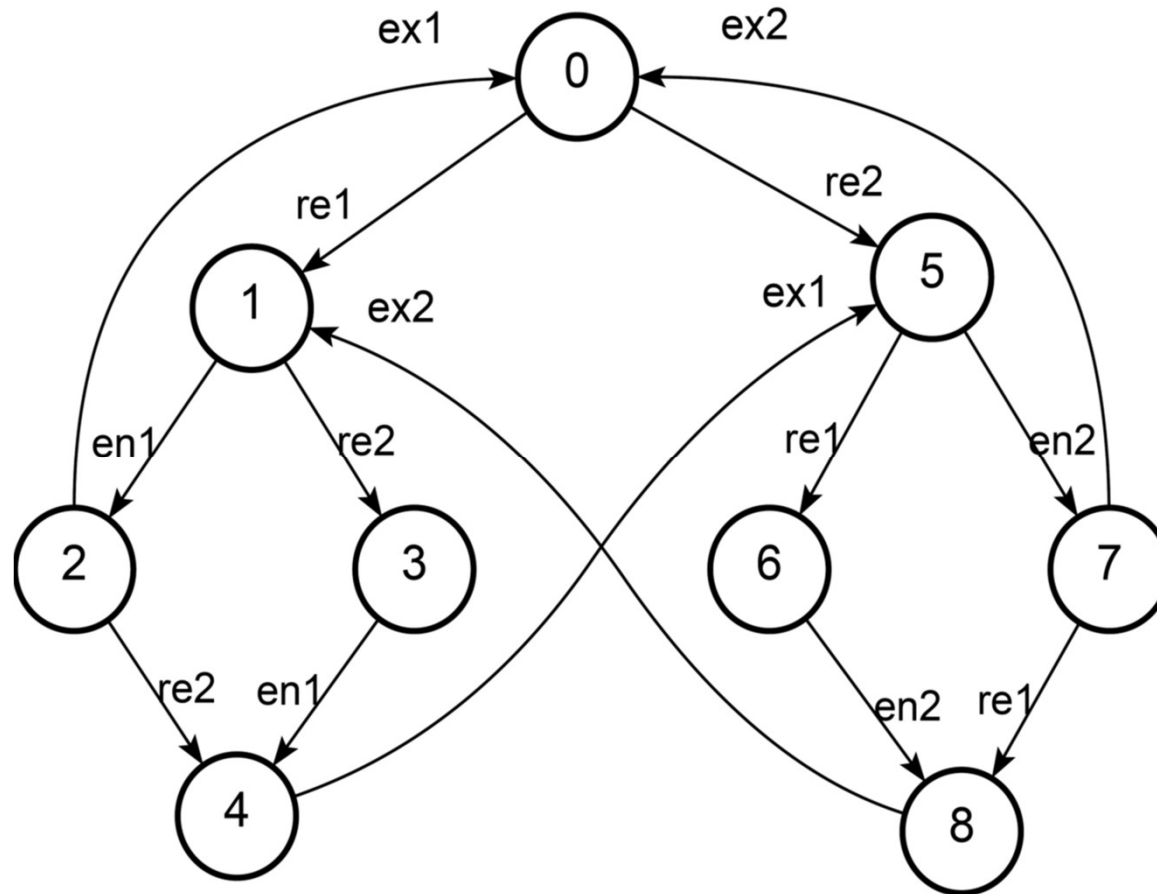
dove $i \geq 1$ e $p \in \Sigma$

Contesto: Il modello LTL (π) è la sequenza infinita di stati/mondi $\pi_1, \pi_2, \pi_3, \dots$
Significato: La proposizione atomica p è vera nello stato π_i (il mondo all'istante i) se e solo se l'azione p è l' i -esima etichetta l_i nella traccia σ .

Se la traccia è finita, le proposizioni dal mondo i in poi (dopo lung. traccia) sono tutte false

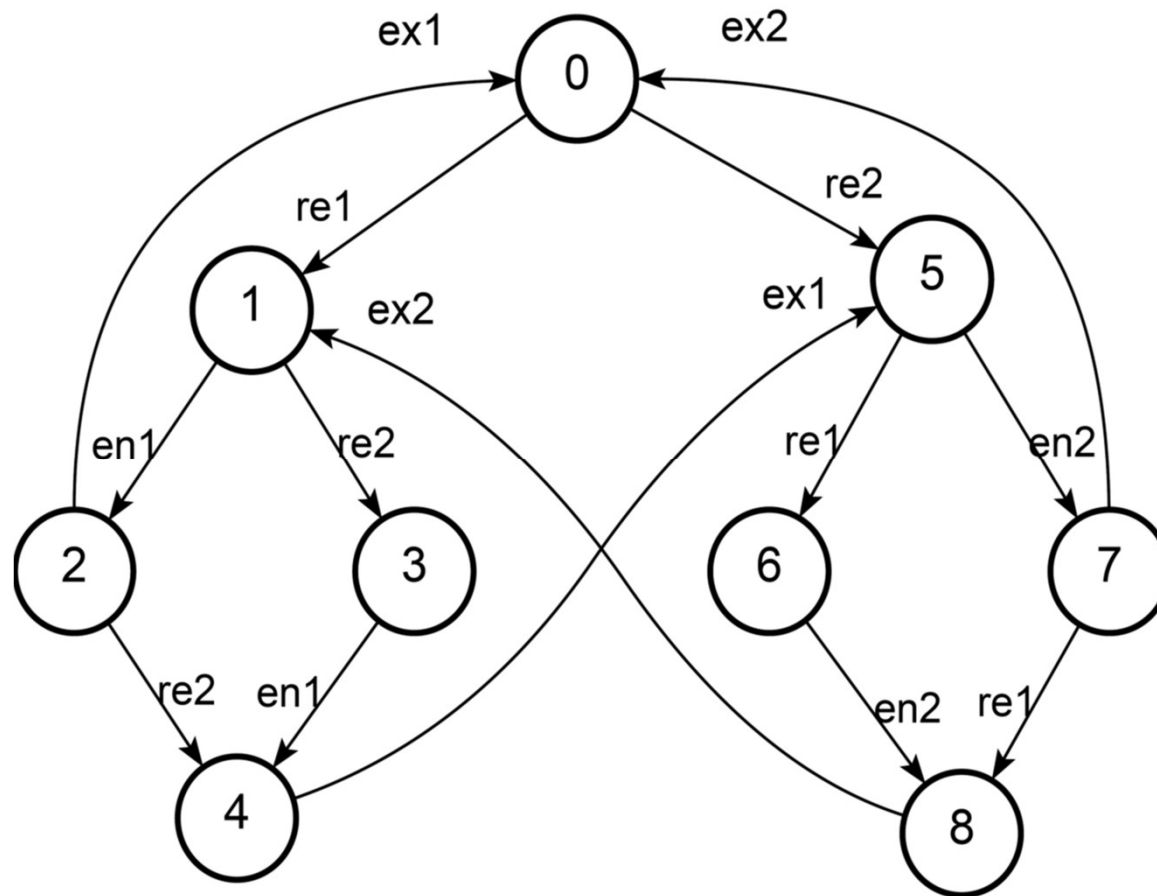
- Un LTS soddisfa una formula ϕ di LTL se i modelli π individuati da **tutte** le sue tracce **massimali** sono tali che: $\pi \models \phi$

Esempio: mutua esclusione



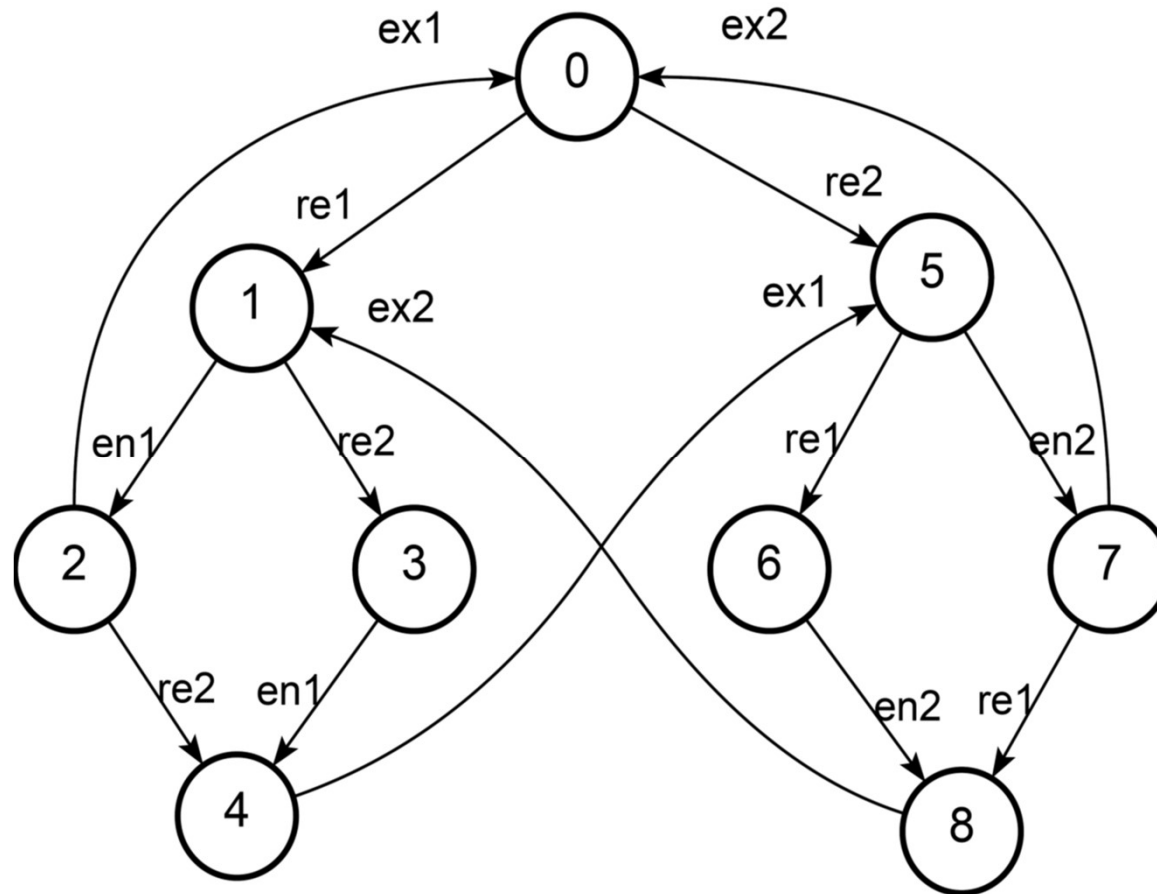
- re1/re2: utente1/utente2 richiedono di entrare in sez. critica
- en1/en2: utente1/utente2 entrano in sezione critica
- ex1/ex2: utente1/utente2 escono da sez. critica

Esempio: mutua esclusione



- Proprietà interessanti (safety):
no contemporanea presenza dei due utenti in
sezione critica

Esempio: mutua esclusione



Come verificare che questa proprietà in questo LTS sia vera? Uso LTS per verificare le infinite tracce (per la verifica, prendo l'etichetta e verifico: esempio, prendo dove ho en1 nel LTS e verifico che la proprietà sia vera)

■ Proprietà interessanti (safety):

$$[](\text{en1} \Rightarrow (-\text{en2} \cup \text{ex1})) \wedge [](\text{en2} \Rightarrow (-\text{en1} \cup \text{ex2}))$$

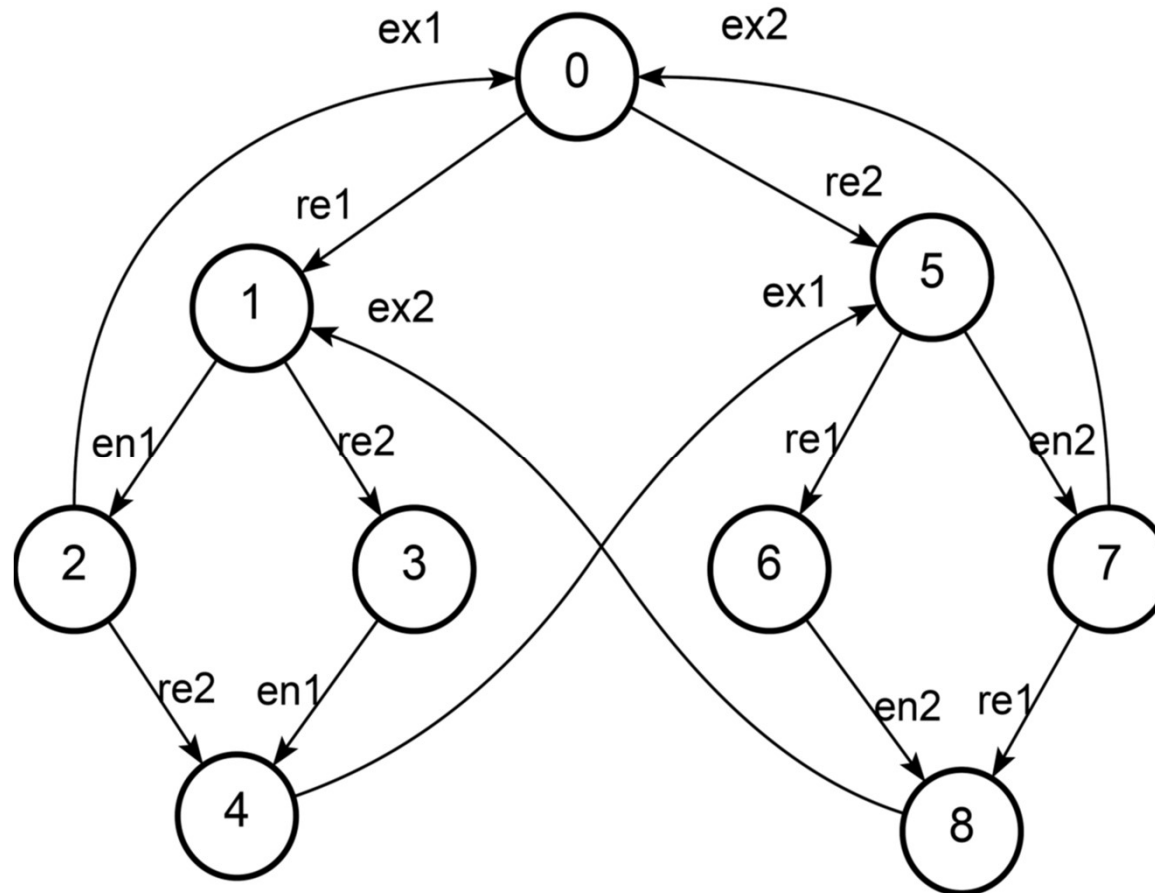
Sottointeso: per ogni traccia LTS è vero il seguente

Adesso ed in futuro se entra Utente1 in sezione critica, ci sarà un mondo futuro in cui lo farò uscire e nei mondi precedenti non faccio entrare Utente2 (stessa cosa dopo l'AND ma al contrario (SIMMETRICO))

Una proprietà di Safety è soddisfatta sull'LTS se la sua formula LTL corrispondente è soddisfatta da tutte le tracce massimali del sistema. In altre parole, una proprietà di safety è vera per un sistema rappresentato da un LTS se nessun malfunzionamento può verificarsi in nessuna esecuzione possibile del sistema stesso.

Esempio: mutua esclusione

Poiché l'LTS è un modello di un sistema, e LTL richiede che la formula sia soddisfatta da tutte le tracce massimali del sistema, il risultato "TRUE" garantisce che:
In ogni possibile sequenza di azioni (traccia) che il sistema può eseguire.
In ogni possibile stato lungo quella traccia.
...la condizione specificata è sempre rispettata.



Il risultato TRUE significa che il Model Checker ha esaminato l'intero spazio degli stati dell'LTS e ha concluso che, a causa della logica di transizione definita nel modello, non esiste nessuna sequenza di scelte che permetta ai due utenti di violare il protocollo di esclusione.

- Proprietà interessanti (safety):

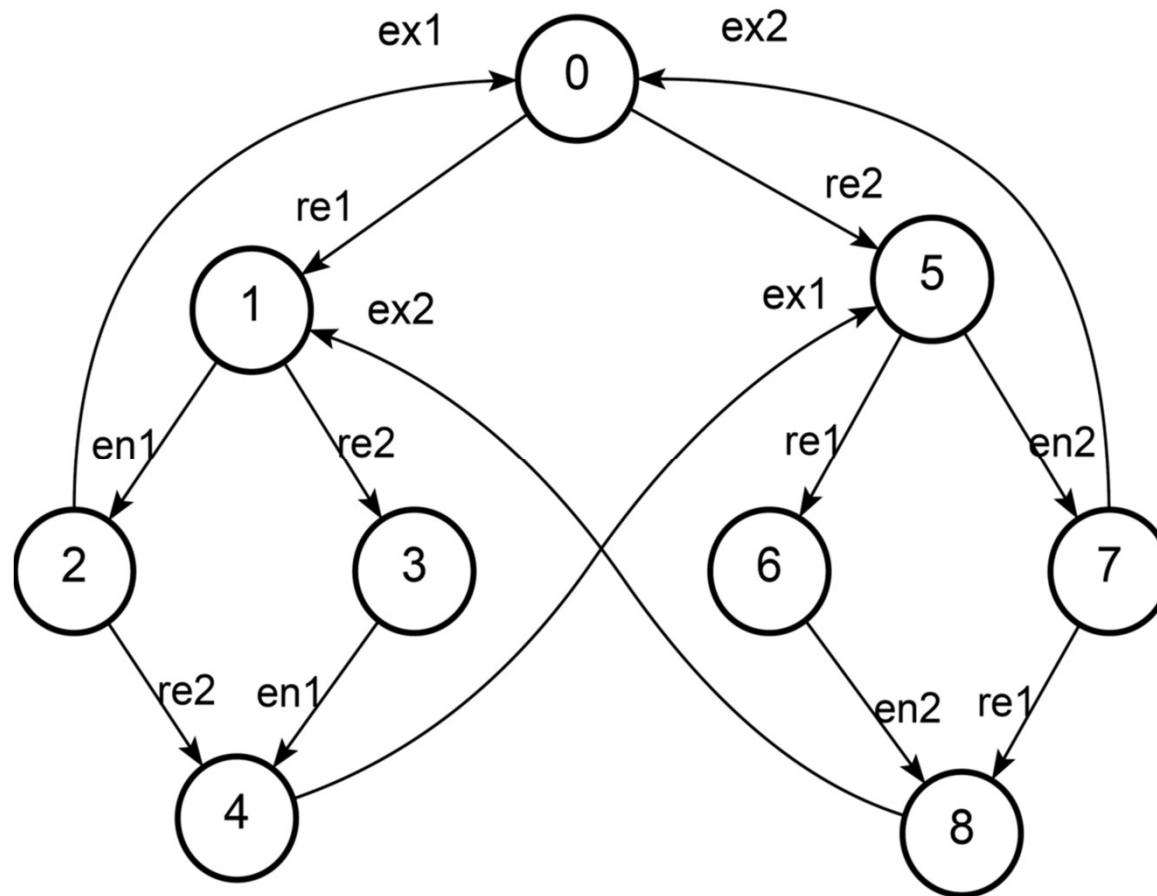
$$[](\text{en1} \Rightarrow (-\text{en2} \cup \text{ex1})) \wedge [](\text{en2} \Rightarrow (-\text{en1} \cup \text{ex2}))$$

TRUE

L'intera formula è una proprietà di Safety (Sicurezza) perché garantisce che "niente di cattivo accadrà mai". L'evento "cattivo" qui è l'ingresso simultaneo. Questa formula impedisce l'ingresso di un processo finché l'altro non ha completato il suo ciclo critico. È una specifica corretta del requisito di mutua esclusione basata sul protocollo di ingresso/uscita.

L'esempio è una formula LTL che esprime una proprietà di Safety, garantendo che i due utenti rispettino un protocollo di accesso che previene l'ingresso simultaneo in una Sezione Critica.

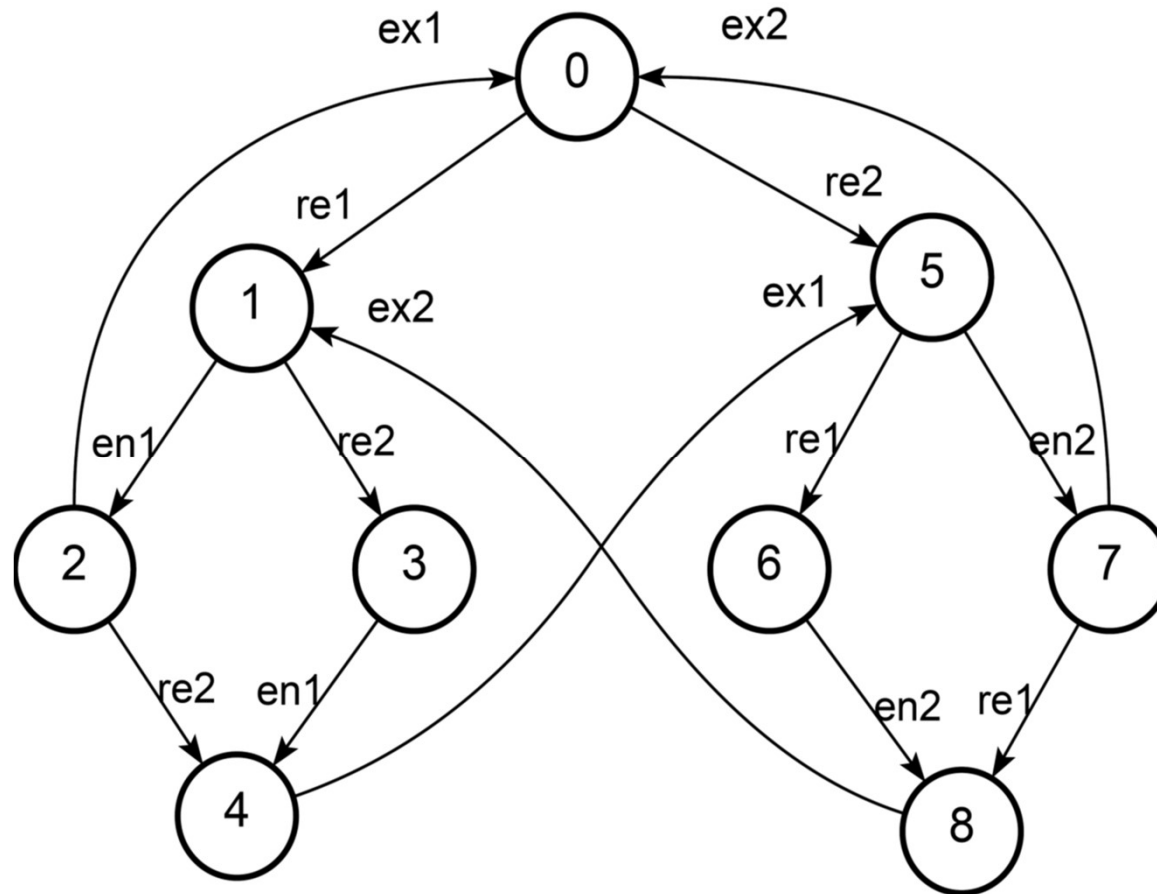
Esempio: mutua esclusione



- **Proprietà interessanti (liveness):** ogni volta che un utente richiede di entrare, successivamente entrerà

Se faccio richiesta, prima
o poi verrà eseguita

Esempio: mutua esclusione

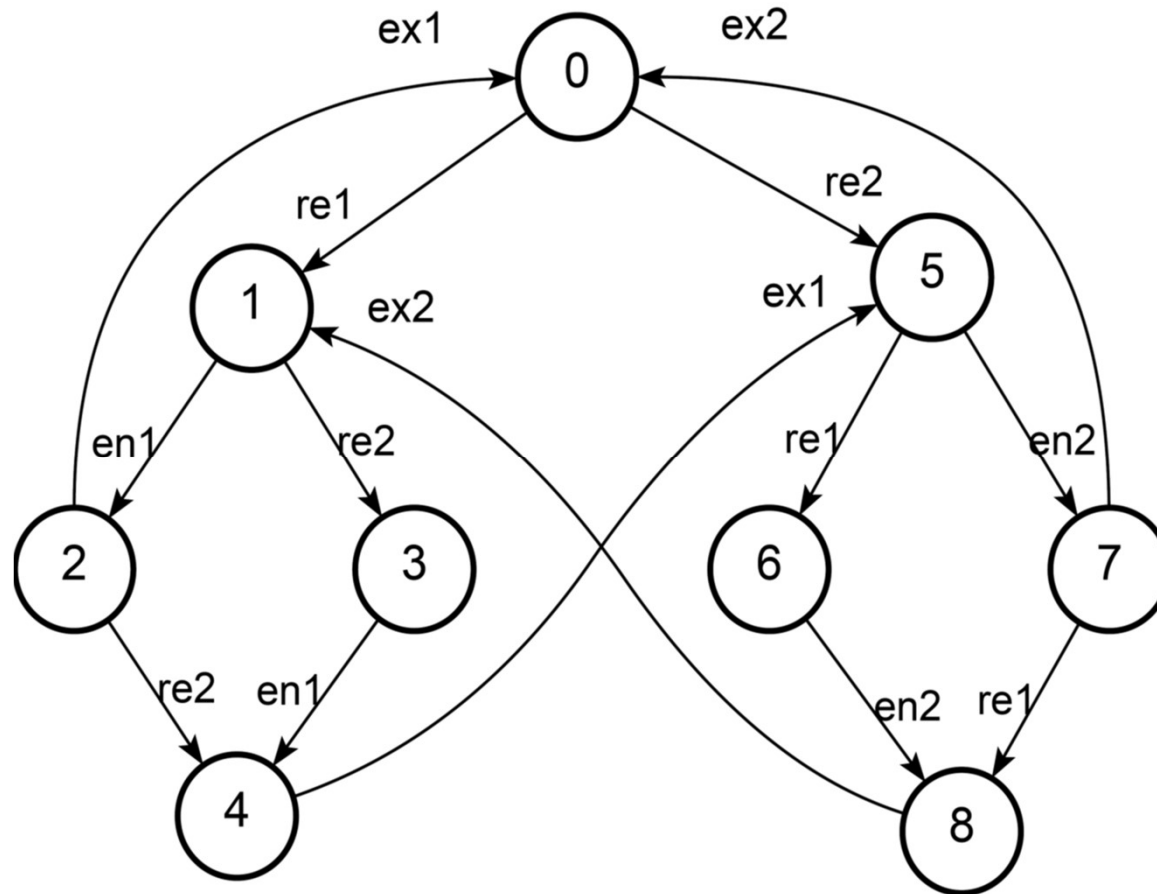


- Proprietà interessanti (liveness):

$\llbracket (re1 \Rightarrow (<> en1)) \wedge \llbracket (re2 \Rightarrow (<> en2))$

Sempre, se il Utente1 richiede l'accesso, allora l'accesso sarà prima o poi garantito (stessa cosa dopo AND: SIMMETRICO).

Esempio: mutua esclusione



L'intera formula esprime una proprietà di Liveness nota come Assenza di Stallo (o Starvation Freedom):

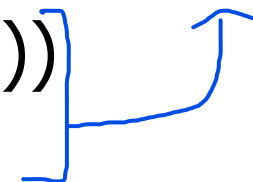
- Garantisce che nessun utente venga bloccato all'infinito (starved) mentre aspetta di accedere alla Sezione Critica.
- Il protocollo deve garantire che, anche se ci sono altri processi in coda o se il sistema è non deterministico, la richiesta di ogni processo venga prima o poi soddisfatta.

Importanza: Mentre la proprietà di Safety garantisce che "niente di cattivo accadrà" (accesso simultaneo), questa proprietà di Liveness garantisce che "qualcosa di buono accadrà" (l'accesso avverrà). Un protocollo di mutua esclusione che garantisce la Safety ma non la Liveness è inutile (ad esempio, un protocollo che impedisce a tutti di entrare risolve la Safety, ma non garantisce la Liveness).

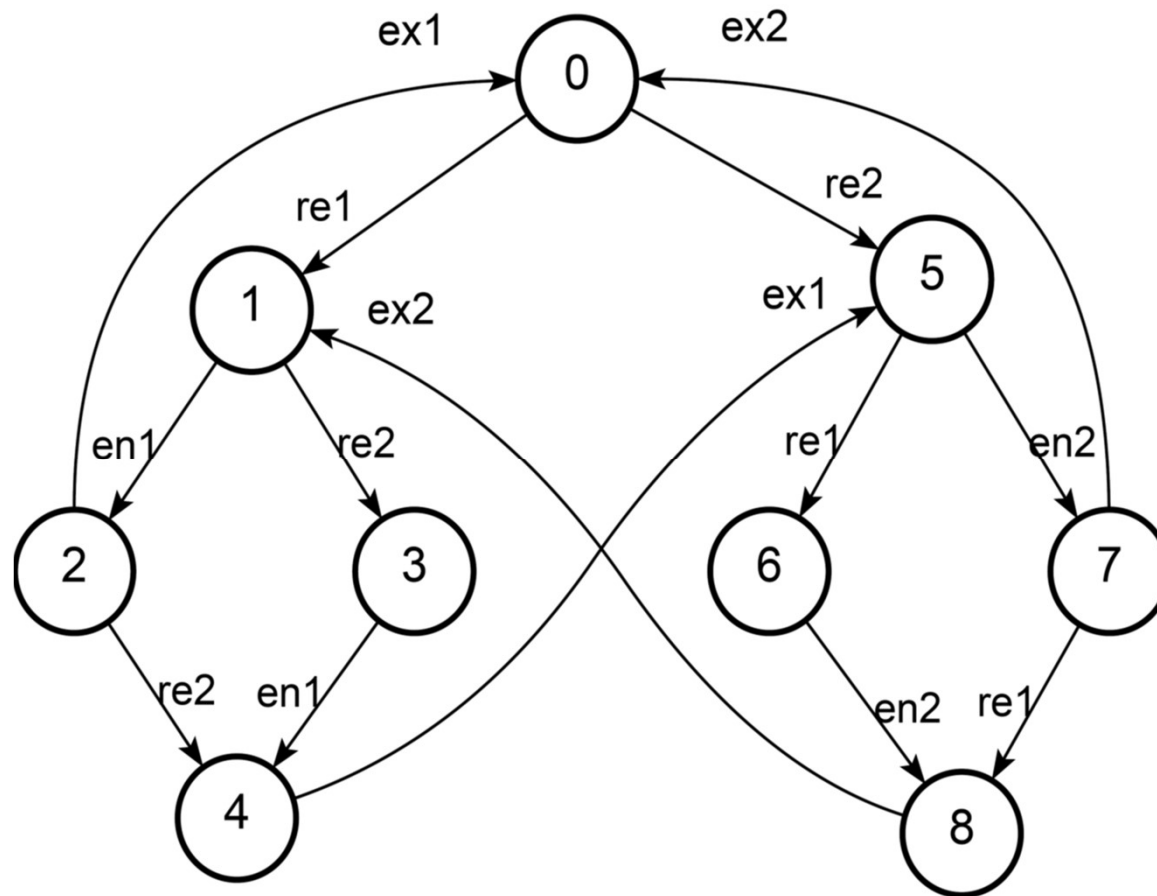
- Proprietà interessanti (liveness):

$$[](\text{re1} \Rightarrow (\langle \rangle \text{en1})) \wedge [](\text{re2} \Rightarrow (\langle \rangle \text{en2}))$$

TRUE

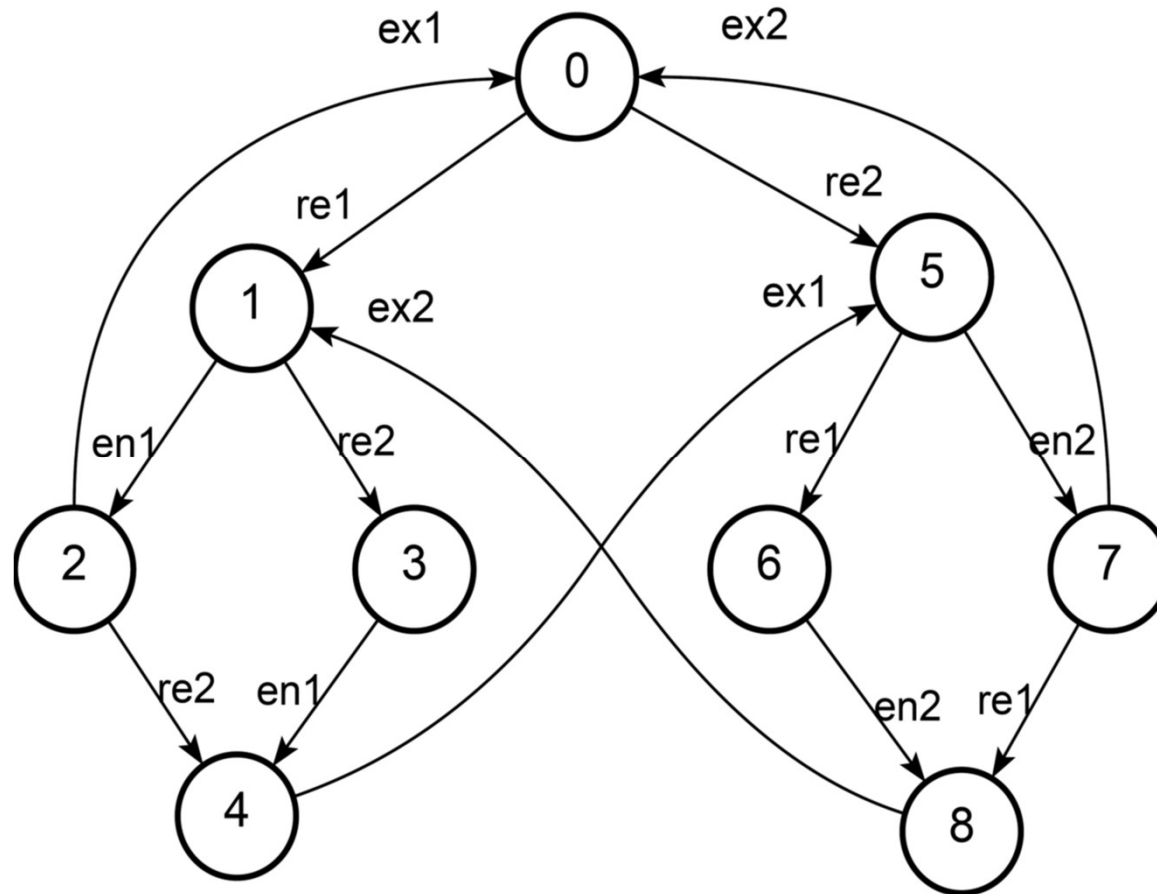


Esempio: mutua esclusione



- **Proprietà interessanti (fairness):**
si noti che si è sempre pronti ad accettare la request di un utente (a meno che non la si abbia già accettata)...

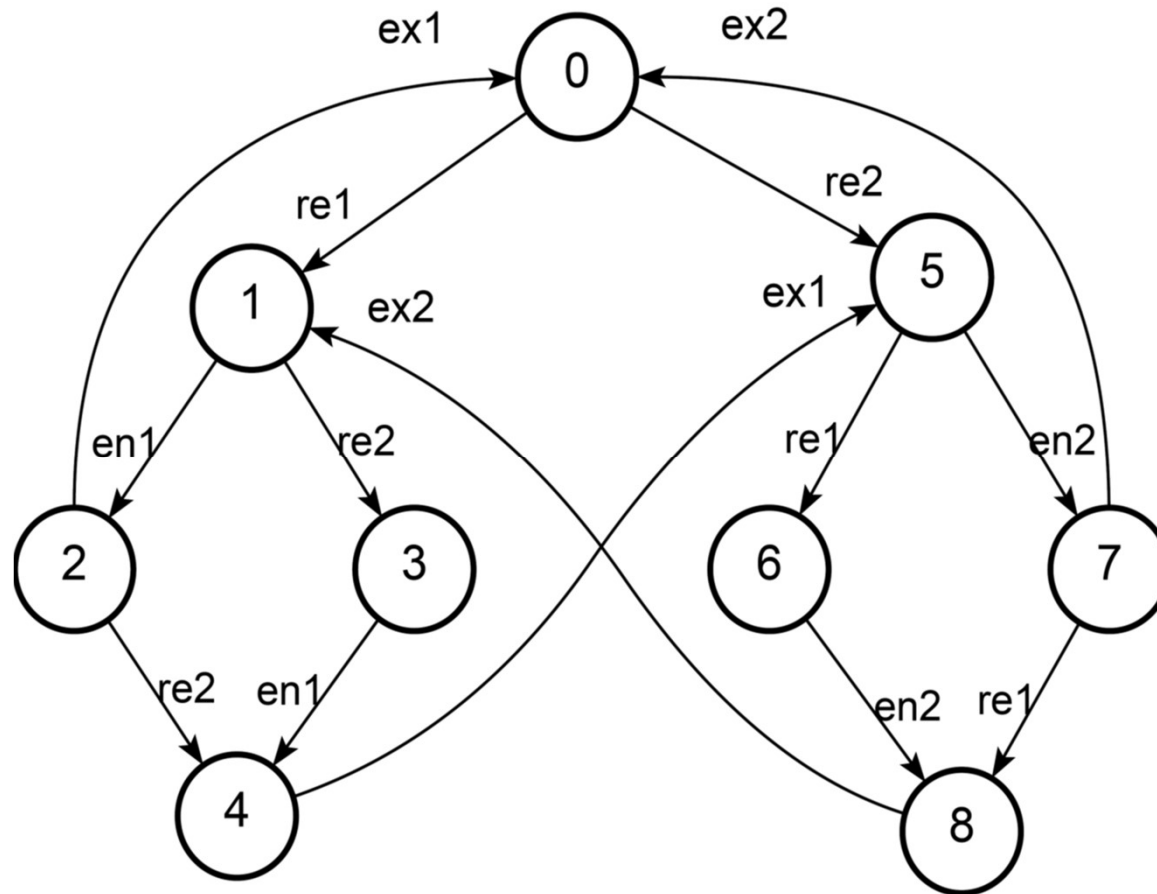
Esempio: mutua esclusione



- Proprietà interessanti (fairness):
...ma siamo sicuri che la request verrà effettivamente accettata?

In realtà, si vede dopo che ho almeno una traccia dove accetto sempre le richieste di un utente e mai dell'altro

Esempio: mutua esclusione



- Proprietà interessanti (fairness):
 $\langle \rangle re1 \wedge \langle \rangle re2$

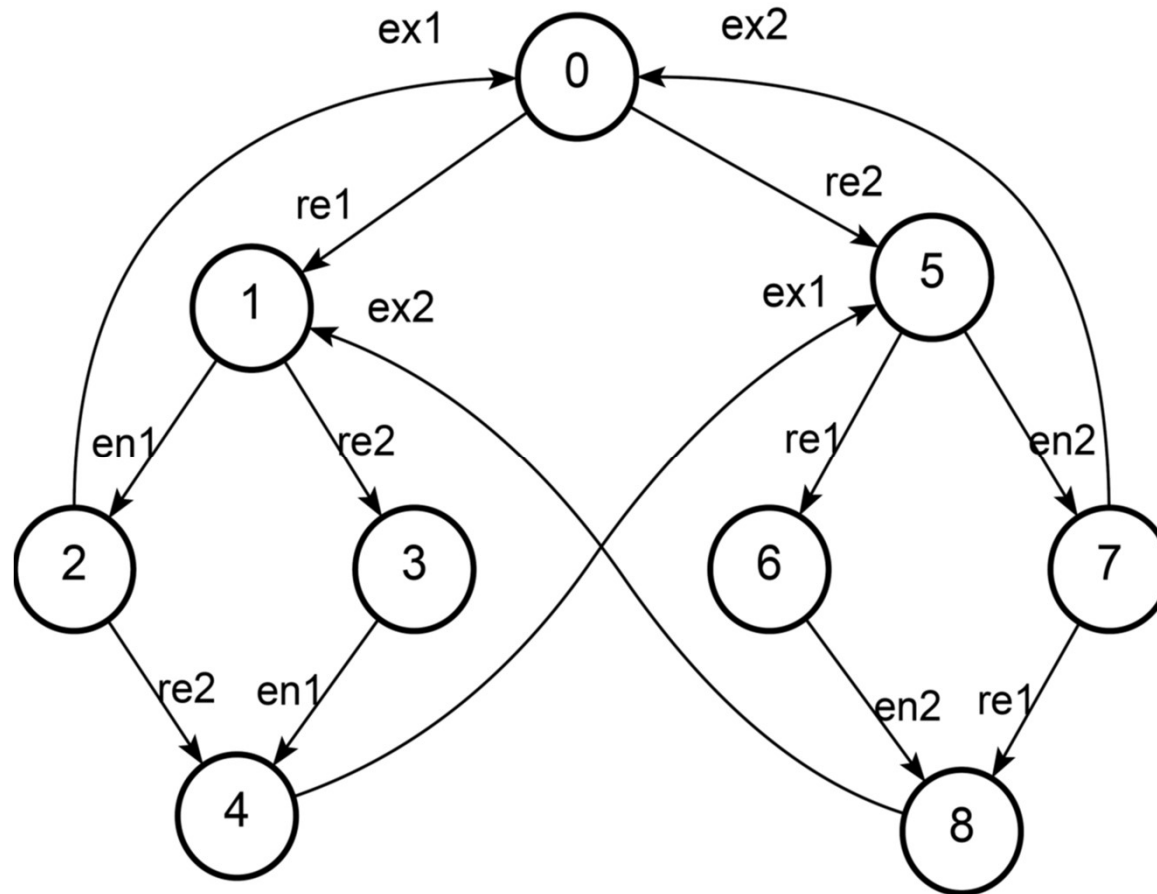
Esempio: mutua esclusione

Quando si deve trovare una traccia che rende una formula LTL (proprietà) false, bisogna guardare solo le etichette che compongono la traccia (quindi non gli stati)

Essendo che le tracce possono essere infinite, per rendere la proprietà falsa con una traccia infinita, scrivo il prefisso di essa

ES: (012)*

* non è la stella di clini perché la stella di clini è per stringhe finite



Il Model Checker ha trovato un controesempio che viola la proprietà di Fairness
- Spiegazione della Traccia: Assumiamo che in qualche momento entrambi i processi siano in richiesta. L'LTS (il sistema) è non deterministico e ha la possibilità di scegliere di servire Utente1 o Utente2. Il controesempio mostra una traccia in cui lo scheduler (il componente non deterministico) sceglie ripetutamente ed all'infinito di dare l'accesso solo all'Utente2. Utente2 richiede, entra, esce e subito dopo richiede di nuovo. L'LTS ha la possibilità di servire Utente1 ma, in questa specifica traccia non equa, sceglie sempre l'azione che ignora le richieste dell'Utente1. Lungo questa traccia usata come controesempio, la richiesta di Utente1 non è mai soddisfatta (<> en1 è falso per Utente1).

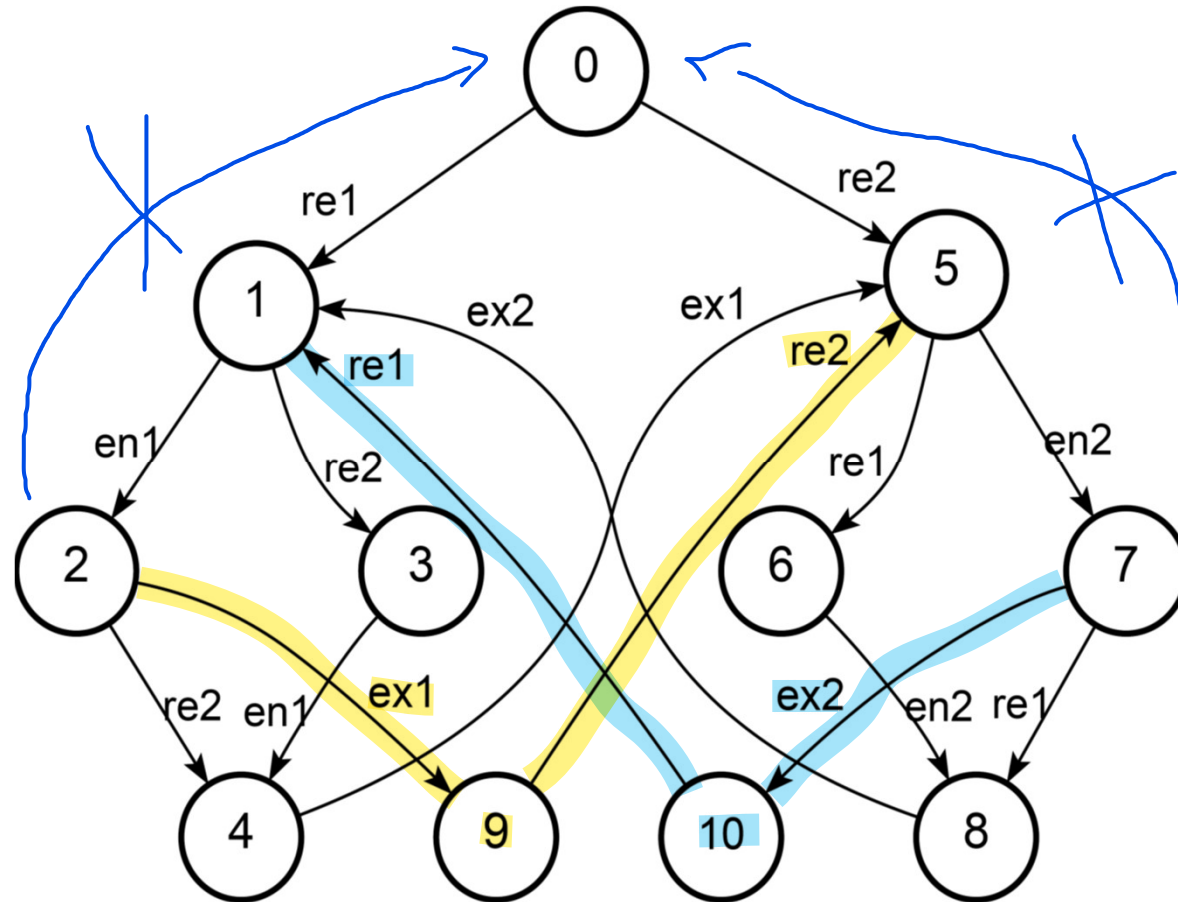
- Proprietà interessanti (fairness):

$\langle \rangle re1 \wedge \langle \rangle re2$

(con controesempio re2-en2-ex2-re2-en2-ex2-...)

FALSE

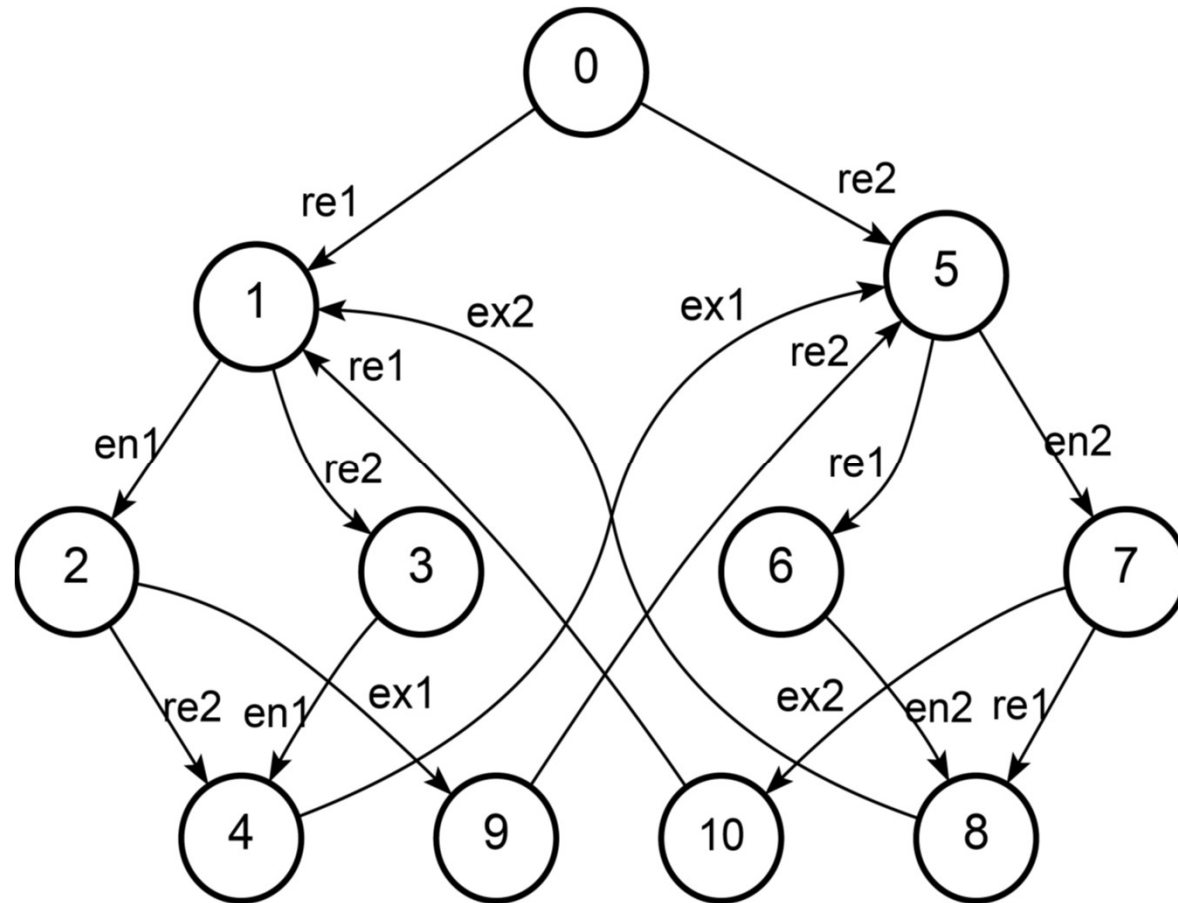
Esempio: mutua esclusione (2)



Nuova versione dell'automa:
forzando l'alternanza fra gli utenti, sarà sempre vero che entrambi potranno eseguire prima o poi la loro request

Creo due nuovi stati per accettare richieste dall'altro utente e rimuovo le transizioni che mi permettevano di accettare solo richieste di un utente

Esempio: mutua esclusione (2)



■ Nuova versione dell'automa:
 $[<>re1 \wedge <>re2]$

TRUE

La differenza è tra una proprietà che deve essere vera una sola volta (a partire dallo stato iniziale) (versione prima dell'automa) e una proprietà che deve essere continuamente re-istanziata e vera per sempre (nuova versione automa).

- 1) Riguarda solo un evento futuro non ripetuto (Deve accadere almeno una volta per ciascuna, a partire da pi_1).
- 2) Riguarda l'infinito futuro e implica la ricorrenza (La possibilità di vedere $re1$ e $re2$ deve essere garantita in ogni futuro istante).

Un Sistema Concorrente è composto da due o più processi che operano (o sembrano operare) simultaneamente e che interagiscono tra loro.

- Interazione: L'interazione può avvenire tramite la condivisione di risorse (come nella mutua esclusione) o tramite lo scambio di messaggi (comunicazione).
- Problemi Chiave: La concorrenza introduce la possibilità di fenomeni complessi e spesso indesiderati, come i deadlock (stallo), le starvation (stallo indefinito) e le race condition (condizioni di gara, in cui l'output dipende dall'ordine non deterministico).

Gli LTS (Labeled Transition Systems) sono il formalismo finale per modellare questi sistemi, ma il problema è come costruire un LTS complesso a partire dalle specifiche dei singoli componenti.

Il Model Checking lavora sull'LTS dell'intero sistema. Il grande vantaggio delle Process Algebra è che esistono algoritmi e strumenti che:

- Prendono le descrizioni algebriche dei singoli processi.
- Applicano l'operatore di composizione parallela.
- Generano automaticamente l'LTS composito (il grafo di accessibilità dell'intero sistema).

Sistemi concorrenti

I sistemi modellati da tali automi (LTS) sono solitamente "sistemi concorrenti":

- Vari processi **interagiscono** (es: i due utenti che interagiscono per accedere alla sezione critica)

Le "process algebra" sono un modo naturale per rappresentare sistemi di questo tipo

- Descrivere **ogni processo** da solo
- **Combinare** le descrizioni di tali processi per ottenere l'intero sistema

Ogni processo è definito come una formula.

l'uso di operatori algebrici per combinare le descrizioni dei processi e ottenere la descrizione dell'intero sistema.

Le Process Algebra sono un linguaggio algebrico ed un insieme di regole per descrivere e ragionare sui sistemi concorrenti in modo modulare e composizionale.

Quando due processi sono composti in parallelo, le loro azioni possono:

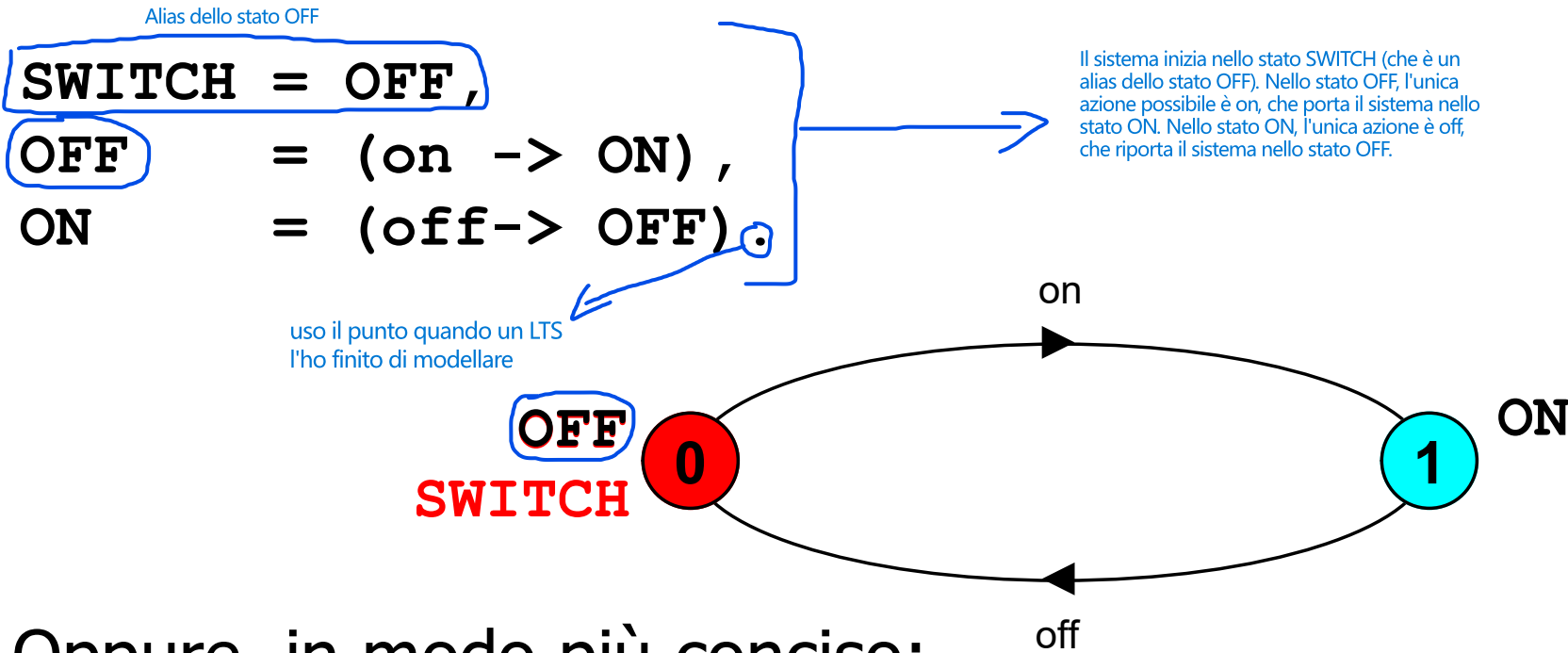
- Interleaving: Se le azioni non sono correlate, possono avvenire in qualsiasi ordine non deterministico. Questo genera il non determinismo nell'LTS finale.
- Sincronizzarsi: Se i processi condividono un'azione, l'azione avviene solo se entrambi sono pronti, modellando la comunicazione o l'accesso a una risorsa condivisa.

Process Algebra

- Esistono tanti tipi di process algebra (CCS, ACP, CSP, pi-calculus, ...)
- Trattiamo FSP (Finite State Processes) usato dal tool LTSA (Labeled Transition System Analyzer)
- In FSP ✓ Un processo è rappresentato come un LTS:
 - si danno **nomi agli stati** del processo
 - per ogni stato si descrivono le **transizioni** che lo stato **può fare**

La Process Algebra (FSP) è il ponte tra l'intenzione del progettista e il modello formale (LTS) su cui viene eseguita la verifica LTL:
Descrizione Modulare: L'ingegnere scrive le specifiche di diversi processi (es. P1, P2) in FSP.
Composizione: L'ingegnere specifica come questi processi interagiscono.
Generazione LTS: LTSA prende questa descrizione composta e la converte in un unico grande LTS (il grafo di accessibilità dell'intero sistema).
Verifica LTL: Il Model Checker valuta le formule LTL (Safety, Liveness, Fairness) su questo LTS generato.

Esempio: interruttore



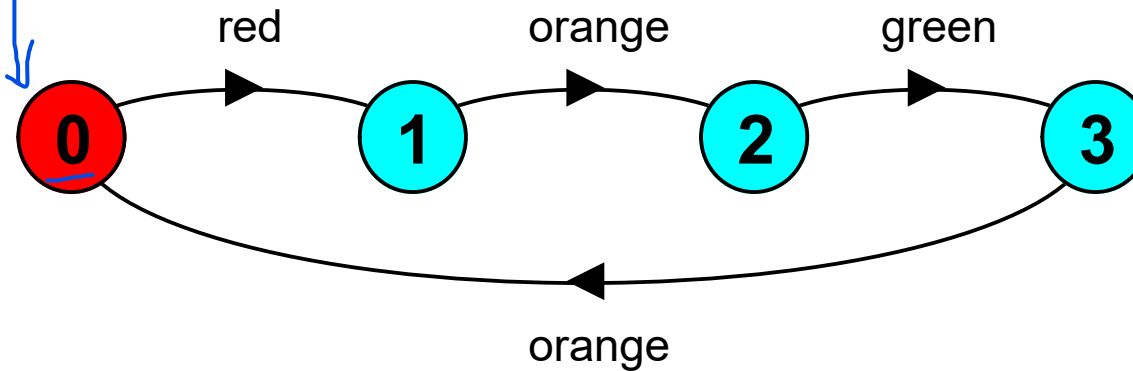
■ Oppure, in modo più conciso:

SWITCH = (on->off->SWITCH) .

Questa notazione crea uno stato "fresh senza nome" (il nodo 1 nel disegno) a metà della sequenza, ma il risultato è un LTS che è ciclico e descrive la sequenza di azioni on e off.

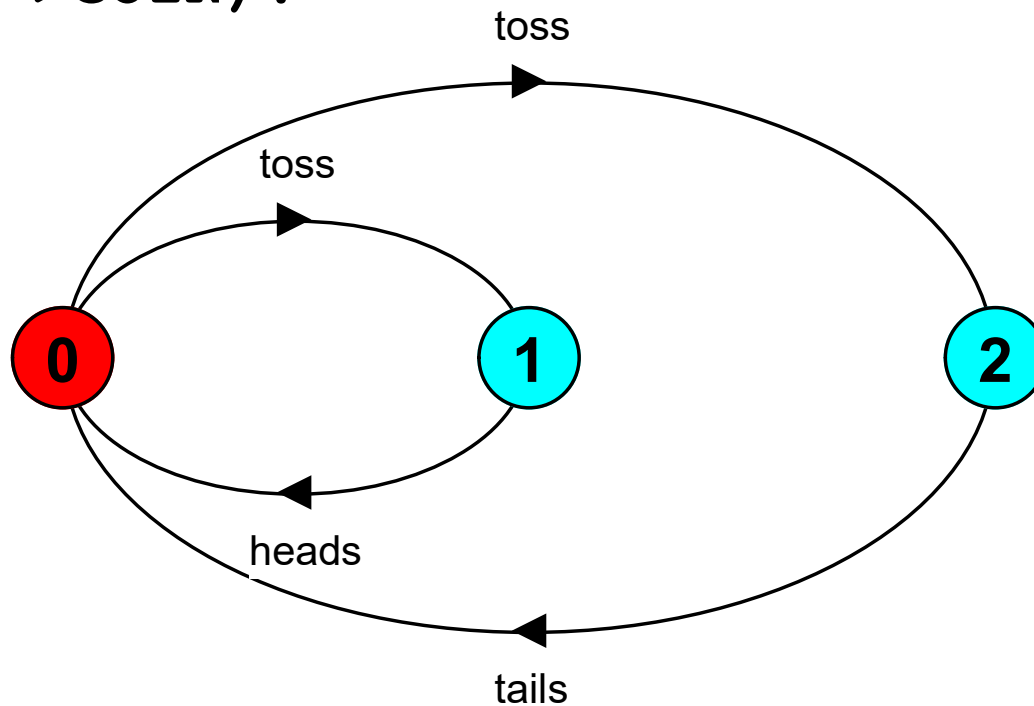
Esempio: semaforo

TRAFFICLIGHT = (red->orange->green->orange
-> TRAFFICLIGHT) .



Esempio: lancio della moneta

⌚ COIN = (toss->HEADS | toss->TAILS) ,
1 HEADS= (heads->COIN) ,
2 TAILS= (tails->COIN) .



La composizione dei processi, nel contesto del Model Checking e delle Algebre di Processo (come FSP), avviene attraverso l'uso di un operatore di composizione parallela. Questo operatore combina gli LTS (Labeled Transition Systems) dei singoli processi per generarne uno unico che rappresenta l'intero sistema concorrente. Il meccanismo di composizione è regolato da due concetti chiave che definiscono l'interazione tra i processi: la Concorrenza (Interleaving) e la Sincronizzazione.

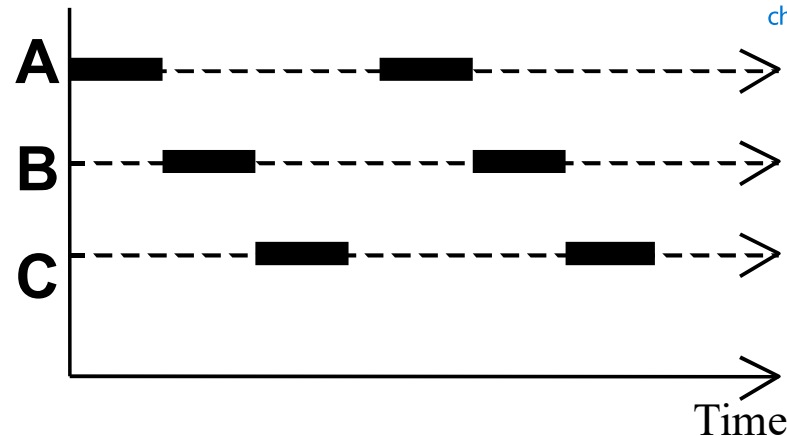
Definizioni

- **Concorrenza (interleaving):**

- **Esecuzione "logicamente" parallela di processi (LTS),
come in un sistema multitasking**

In pratica, si eseguono i processi in modo parallelo (se a disposizione più core) e non deterministico

Significato: Le azioni di processi diversi vengono eseguite in sequenza in un ordine non deterministico.
Modellazione: Se due processi, A e B, sono composti in parallelo e hanno azioni distinte (a e b), l'LTS composito includerà transizioni che permettono sia la sequenza 'a seguita da b' che 'b seguita da a'.



- **Sincronizzazione:**

- **Esecuzione "fisicamente" contemporanea di azioni**

Meccanismo: Se due processi condividono un'azione con la stessa etichetta, l'azione può avvenire solo quando entrambi i processi sono pronti ad eseguirla.
Azione Congiunta: Nell'LTS composito, l'esecuzione di quell'azione è rappresentata da una singola transizione che fa avanzare entrambi i processi simultaneamente.
Vantaggio: Questo garantisce che la comunicazione (o il passaggio di un token di mutua esclusione) avvenga in modo controllato e atomico.

Parallelismo: interleaving e sincronizzazione

- **Interleaving:** Concetto: L'esecuzione è logicamente parallela ma fisicamente sequenziale (un'azione alla volta). Ordine: L'ordine in cui le azioni non correlate vengono eseguite è arbitrario e non deterministico.
 - Transizioni **specifiche** di un processo (cioè assenti in altri processi) vengono eseguite in "interleaving"
 - **Interleaving:** una azione alla volta in **ordine arbitrario** (nessuna assunzione sulla velocità relativa dei processi)
- **Sincronizzazione:** Concetto: Prevede l'esecuzione simultanea di una azione da parte di più LTS. L'azione si verifica solo quando tutti i processi che la condividono sono pronti ad eseguirla.
 - Su **etichette comuni** (nell'alfabeto degli LTS eseguiti in parallelo) è prevista "sincronizzazione"
 - **Sincronizzazione:** esecuzione **simultanea** di una azione da parte di più LTS

Questo esempio dimostra che, se non ci sono azioni condivise, il numero di possibili esecuzioni del sistema cresce esponenzialmente in base al grado di non determinismo (cioè, il numero di possibili interleaving).

Esempio: composizione parallela tramite interleaving

Composizione parallela basata sull'Interleaving (Interfoliazione), che avviene quando i processi non condividono azioni, ovvero hanno alfabeti disgiunti.

ITCH = (scratch->STOP) .

CONVERSE = (think->talk->STOP) .

Poiché gli alfabeti {scratch} e {think,talk} sono disgiunti (non ci sono azioni in comune), non ci sono sincronizzazioni. Le azioni vengono eseguite in interleaving.

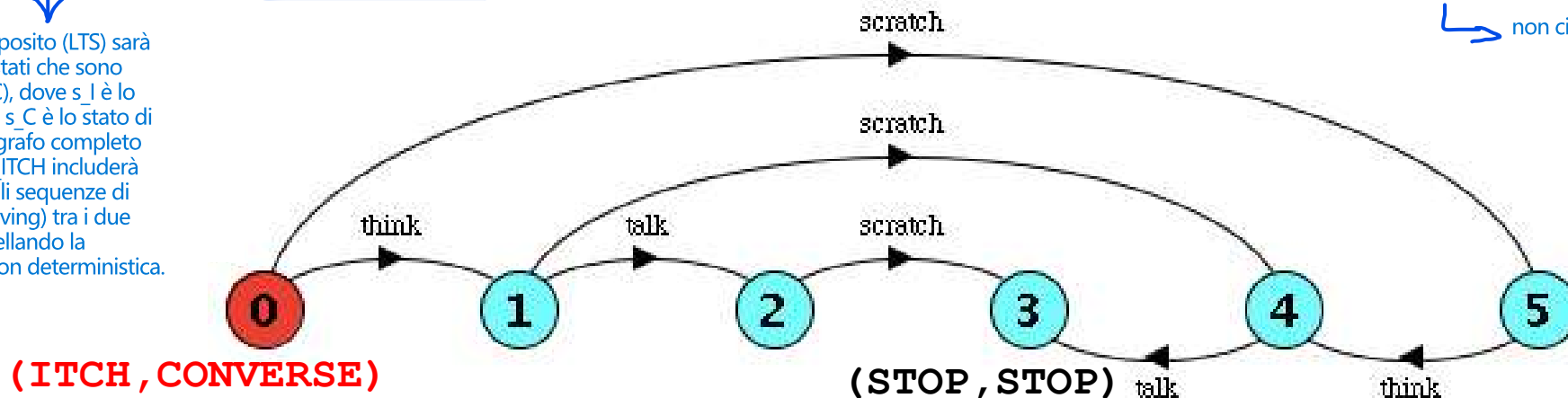
Operatore di
Composizione
Parallela

$||$ CONVERSE_ITCH = (ITCH $||$ CONVERSE) .

- Avendo **alfabeti disgiunti**, le azioni vengono eseguite in interleaving (STOP è stato di blocco):

Il sistema composito (LTS) sarà costruito con stati che sono coppie (s_I, s_C), dove s_I è lo stato di ITCH e s_C è lo stato di CONVERSE. Il grafo completo di CONVERSE_ITCH includerà tutte le possibili sequenze di azioni (Interleaving) tra i due processi, modellando la concorrenza non deterministica.

non ci sono transizioni in uscita



Esempio: composizione parallela con sincronizzazione

MAKER = (make->ready->used->MAKER) .

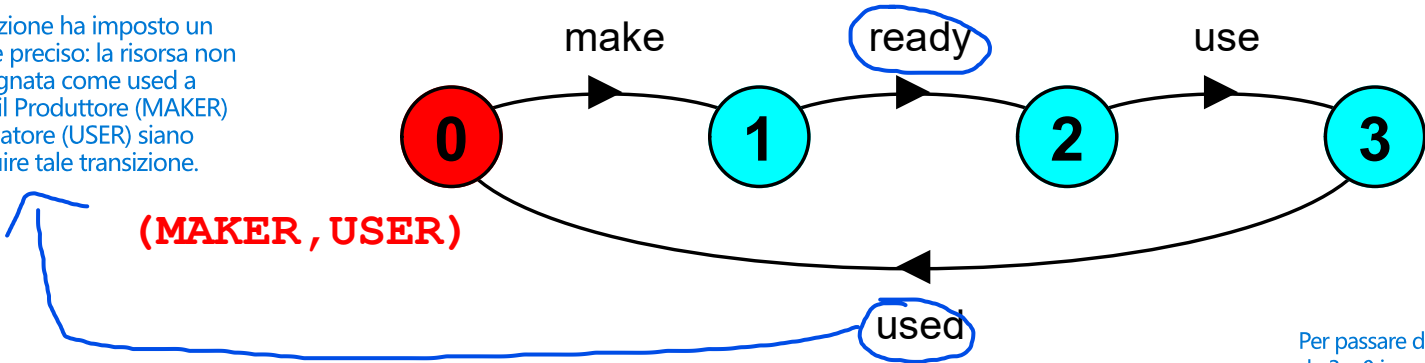
USER = (ready->use->used->USER) .

||MAKER_USER = (MAKER || USER) .

- Azioni Condivise (Sincronizzate): Le etichette ready e used sono presenti in entrambi gli alfabeti. Per la regola della composizione parallela, queste transizioni devono sincronizzarsi.
- Azioni in Interleaving (Individuali): Le azioni make (solo in MAKER) e use (solo in USER) sono specifiche e avvengono per interleaving.

- “ready” e “used” sono in entrambi gli alfabeti,
quindi tali transizioni devono sincronizzarsi:

La sincronizzazione ha imposto un ordine causale preciso: la risorsa non può essere segnata come used a meno che sia il Produttore (MAKER) che il Consumatore (USER) siano pronti a eseguire tale transizione.



Grazie alla sincronizzazione, l'LTS composto non ha scelte non deterministiche (non c'è ramificazione) e descrive un unico cammino ciclico

Queste due regole (Sincronizzazione e Interleaving) definiscono tutte le transizioni possibili dell'LTS composito. Il Model Checker esplora questo grafo di stati generato per verificare le proprietà LTL.

Formalmente

insieme di stati alfabeto delle transizioni funzione di transizione stati iniziali

Dati due LTS $A_i = (Q_i, \Sigma_i, \delta_i, q_0^i)$, la loro **composizione parallela** $A_1 || A_2$ è l'LTS $(Q_1 \times Q_2, \Sigma_1 \cup \Sigma_2, \delta, (q_0^1, q_0^2))$, dove δ è definita:

Si verifica per ogni transizione a che appartiene all'intersezione degli alfabeti

- per ogni $a \in \Sigma_1 \cap \Sigma_2$ (caso della **sincronizzazione**) Prodotto cartesiano degli stati raggiungibili da 'p con a' e da 'q con a'

$$\delta((p, q), a) = \delta_1(p, a) \times \delta_2(q, a)$$

cioè se $p \xrightarrow{a} p'$ e $q \xrightarrow{a} q'$ allora $(p, q) \xrightarrow{a} (p', q')$

Se il Processo1 può passare da p a p' tramite a, ed il Processo2 può passare da q a q' tramite a, allora il sistema composito può passare dallo stato combinato (p, q) al nuovo stato combinato (p', q') tramite l'unica azione a.

- per ogni $a \notin \Sigma_1 \cap \Sigma_2$ (caso dell'**interleaving**) Prodotto cartesiano dove uno dei due LTS non scorre, cioè rimane fermo su quello stato in cui si trova (si muove uno solo)

$$\delta((p, q), a) = \delta_1(p, a) \times \{q\} \quad \text{se } a \in \Sigma_1$$

cioè se $p \xrightarrow{a} p'$ allora $(p, q) \xrightarrow{a} (p', q)$

Se solo A1 può eseguire a e passa da p a p', lo stato di A2 (q) rimane lo stesso.

$$\delta((p, q), a) = \{p\} \times \delta_2(q, a) \quad \text{se } a \in \Sigma_2$$

cioè se $q \xrightarrow{a} q'$ allora $(p, q) \xrightarrow{a} (p, q')$

Se solo A2 può eseguire a e passa da q a q', lo stato di A1 (p) rimane lo stesso.

Proprietà della composizione parallela

- La composizione parallela è **commutativa** e **associativa**
 - L'LTS di $(A_1 || A_2)$ è lo stesso di $(A_2 || A_1)$
 - L'LTS di $(A_1 || A_2) || A_3$ è lo stesso di $A_1 || (A_2 || A_3)$
- Possiamo quindi denotare la **composizione parallela di multipli LTS** $A_1, A_2, \dots, A_n$ scrivendo semplicemente $A_1 || A_2 || \dots || A_n$ senza parentesi

Esempio: composizione parallela multipla

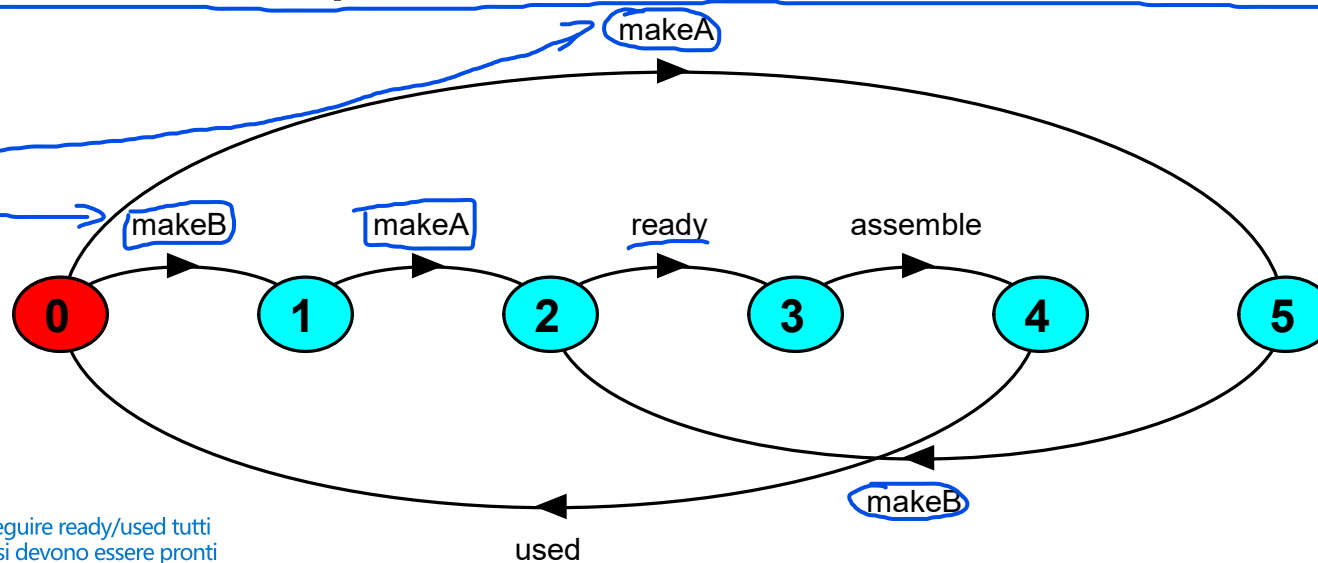
Il sistema FACTORY modella due produttori (MAKE_A, MAKE_B) e un assemblatore/consumatore (ASSEMBLE). Le azioni condivise impongono un rigoroso coordinamento tra i tre componenti.

MAKE_A = (makeA->ready->used->MAKE_A) 
 MAKE_B = (makeB->ready->used->MAKE_B) 
 ASSEMBLE = (ready->assemble->used->ASSEMBLE) 

|| FACTORY = (MAKE_A || MAKE_B || ASSEMBLE) .

- "ready" e "used" sono in tutti e 3 gli alfabeti,
quindi i tre processi devono sincronizzarsi:

Essendo che, prima di eseguire 'ready', abbiamo due transizioni da eseguire in interleaving, dobbiamo considerare i due casi in cui prima eseguito makeA e poi



Prima di eseguire ready/used tutti e 3 i processi devono essere pronti ad eseguire quella transizione

Il controller è il cuore del protocollo di Mutua Esclusione:

- Sincronizzazione Implicita: Le azioni requestX, enterX, e exitX sono condivise tra il CONTROLLER e l'utente corrispondente (USERX). Questo significa che tutte queste azioni devono avvenire in sincronizzazione a due vie.
- Controllo Esclusivo: L'operatore '|' (OR) all'interno del CONTROLLER è cruciale: in qualsiasi momento, il controller può scegliere di avviare o il ciclo di P1 (request1...) o il ciclo di P2 (request2...).
- Ordine Rigido: Una volta che il controller sceglie request1, deve completare l'intera sequenza enter1 -> exit1 prima di poter tornare allo stato iniziale e prendere una nuova decisione.

Garantisce Safety, Liveness, ma non Fairness
(perché posso avere delle tracce che eseguono solo P1 o solo P2, senza mai eseguire l'altro)

Esempio: mutua esclusione

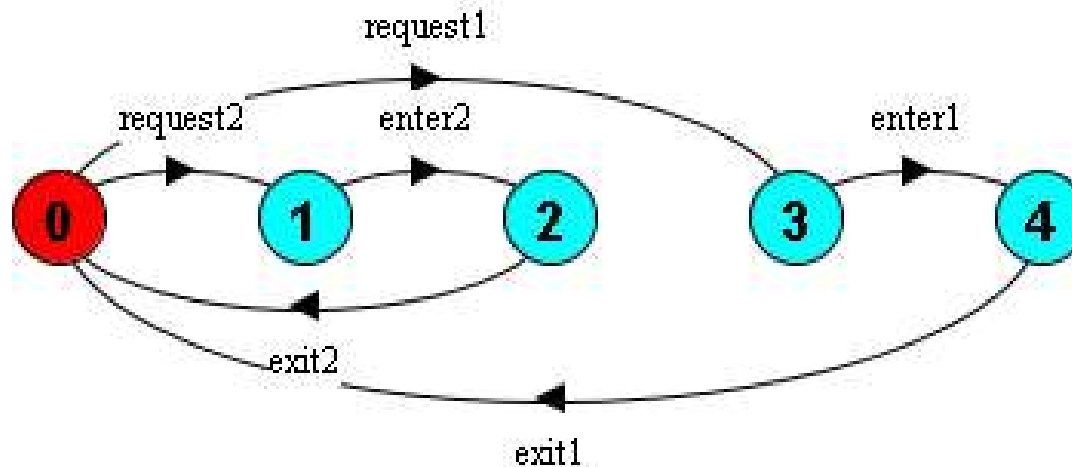
I processi utente sono sistemi ciclici identici che definiscono le azioni che desiderano eseguire

```
USER1 = (request1 -> enter1 -> exit1 -> USER1 ) .  
USER2 = (request2 -> enter2 -> exit2 -> USER2 ) .
```

```
CONTROLLER = (request1->enter1->exit1->CONTROLLER |  
               request2->enter2->exit2->CONTROLLER) .
```

Il sistema MUTUAL_EXCLUSION è composto da due utenti concorrenti (USER1, USER2) e un supervisore centrale (CONTROLLER).

```
||MUTUAL_EXCLUSION = ( USER1 || USER2 || CONTROLLER ) .
```



Esempio: mutua esclusione (2)

I processi utente rimangono semplici e ciclici, definendo solo le azioni che desiderano eseguire

```
USER1 = (request1 -> enter1 -> exit1 -> USER1) .  
USER2 = (request2 -> enter2 -> exit2 -> USER2) .
```

Il nuovo CONTROLLER è un LTS molto più sofisticato, progettato per gestire lo stato di "turno" e le richieste in sospeso.

```
CONTROLLER = ( request1 -> TURN1  
                | request2 -> TURN2 ) ,
```

```
TURN1 = ( request2 -> enter1 -> exit1 -> TURN2  
          | enter1 -> ( request2 -> exit1 -> TURN2  
                      | exit1 -> CONTROLLER  
                      )  
          ) ,
```

```
TURN2 = ( request1 -> enter2 -> exit2 -> TURN1  
          | enter2 -> ( request1 -> exit2 -> TURN1  
                      | exit2 -> CONTROLLER  
                      )  
          ) .
```

```
|| MUTUAL_EXCLUSION = ( USER1 || USER2 || CONTROLLER ) .
```

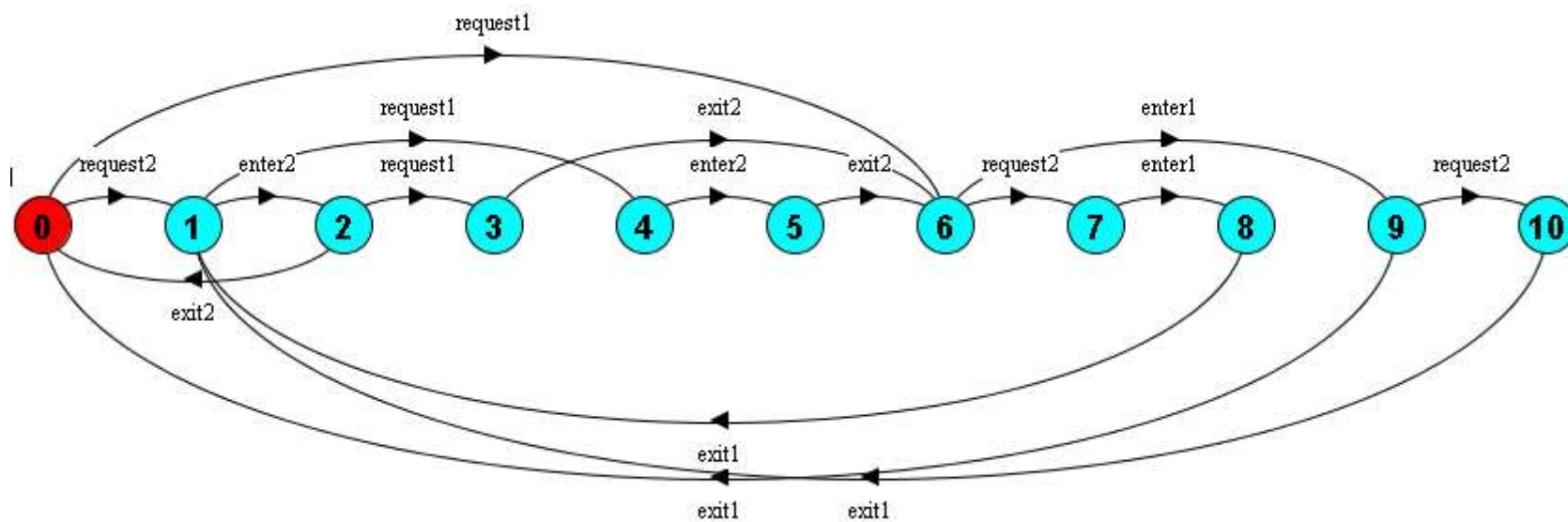
Questo stato modella due scenari principali:

- Scenario con Richiesta in Sospeso (Starvation Prevention): Se, mentre P1 ha la priorità, P2 fa una richiesta (request2), il CONTROLLER ricorda questa richiesta (entrando in questo ramo). Il controller forza P1 a completare il suo ciclo critico prima di passare a TURN2 (dove P2 avrà la priorità). Questa sequenza assicura che la richiesta di P2 non venga persa.
- Scenario Senza Richiesta in Sospeso: P1 entra subito, poiché P2 non ha richiesto. Una volta che P1 è dentro, se P2 richiede, il controller memorizza la richiesta di P2 prima che P1 esca. Se P1 esce senza che P2 abbia richiesto, il controller torna allo stato iniziale.

Lo stato TURN2 è simmetrico a TURN1, dando priorità al USER2.

Oltre a Safety e Liveness, viene garantito anche Fairness: Fairness (Liveness): Se un utente richiede l'accesso mentre l'altro è in esecuzione, lo stato TURNX memorizza la richiesta in sospeso (requestY) e garantisce che il controllo passi al richiedente in sospeso non appena l'utente corrente esce. Questo protocollo implementa un meccanismo di coda implicita o alternanza con priorità, eliminando le tracce ingiuste (il controesempio precedente) dall'LTS.

Esempio: mutua esclusione (2)



Meccanismo e Scopo

- Limitazione dell'Alfabeto: La sintassi $P @ \{a_1, \dots, a_x\}$ limita l'alfabeto di P alle sole azioni specificate nell'insieme $\{a_1, \dots, a_x\}$.

- Conversione in Tau: Tutte le altre azioni di P che non sono elencate nel set di hiding vengono trasformate nell'azione locale tau.

- tau (Azione Interna): L'azione tau rappresenta una transizione interna (non osservabile) che non può sincronizzarsi con nessun altro processo.

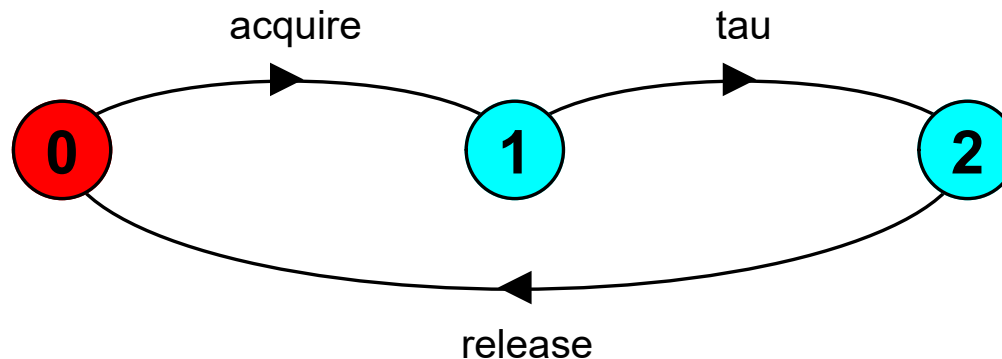
- Scopo: Lo scopo principale è evitare che tali azioni vengano sincronizzate con altri processi, trasformando l'interazione da pubblica a privata. Questo è cruciale per astrarre i dettagli implementativi e ridurre la dimensione del grafo degli stati.

Hiding

L'operatore di Hiding, denotato con $@ \{a_1, \dots, a_x\}$, viene applicato a un processo P per limitare l'alfabeto di azioni visibili ad altri processi.

- Con " $P @ \{a_1, \dots, a_x\}$ " si **limita l'alfabeto** di P alle azioni $\{a_1, \dots, a_x\}$
 - **Altre azioni** vengono trasformate in azione locale "**tau**"
 - In questo modo si **evita** che tali azioni vengano **sincronizzate** con altri processi (in pratica, le transizioni tau non vengono sincronizzate ed esposte con le transizioni esposte dagli altri processi)

$USER = (\underline{acquire} \rightarrow \underline{use} \rightarrow \underline{release} \rightarrow USER) @ \{\underline{acquire}, \underline{release}\} .$



Questa formalizzazione è cruciale per la verifica, in quanto:

- Astrae il modello: Riduce la complessità del sistema rendendo le azioni irrilevanti interne.
- Mantiene la correttezza: Lo stato del sistema p' raggiunto da una transizione nascosta è mantenuto, ma l'azione che ha causato il cambiamento di stato non è più visibile.

Formalmente

Dato un LTS $A = (Q, \Sigma, \delta, q_0)$, l'LTS ottenuto **limitando il suo alfabeto a** $\Sigma' \subseteq \Sigma$ è $A@ \Sigma' = (Q, \Sigma', \delta', q_0)$, dove δ' è definita:

- per l'**azione speciale tau** (caso di azioni che **si nascondono**)

$$\delta'(p, \text{tau}) = \bigcup_{a \notin \Sigma'} \delta(p, a)$$

Le transizioni eseguite tramite azioni a che non sono in Σ' vengono convertite in transizioni tau.

cioè se $p \xrightarrow{a} p' \wedge a \notin \Sigma'$ allora in $A@ \Sigma'$ si ha $p \xrightarrow{\text{tau}} p'$

L'azione tau modella l'esecuzione di un'azione interna al processo, non osservabile e non sincronizzabile dall'esterno.

- per ogni $a \in \Sigma'$ (caso di azioni che **non si nascondono**)

$$\delta'(p, a) = \delta(p, a)$$

Le transizioni eseguite tramite azioni a in Σ' rimangono inalterate.

cioè se $p \xrightarrow{a} p' \wedge a \in \Sigma'$ allora in $A@ \Sigma'$ si ha $p \xrightarrow{a} p'$

Queste azioni mantengono la loro identità e sono disponibili per l'interleaving o la sincronizzazione con altri processi esterni.

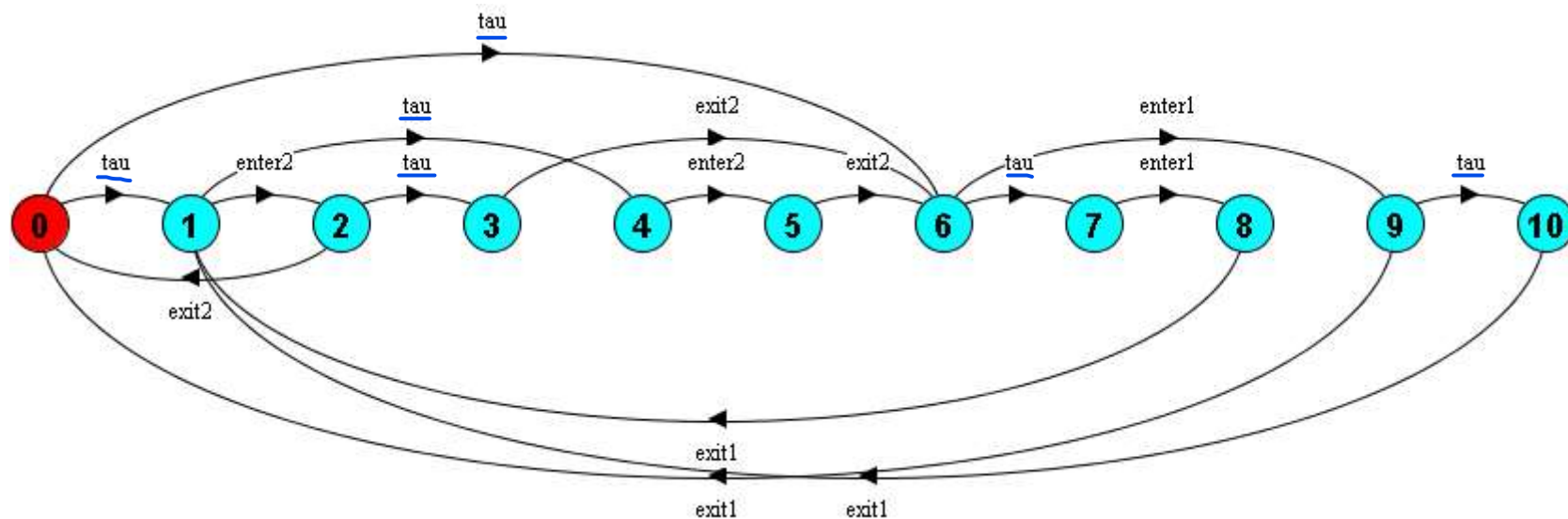
Esempio: mutua esclusione (3)

- Possiamo nascondere le azioni di richiesta

```
||MUTUAL_EXCLUSION =  
  ( USER1 || USER2 || CONTROLLER )@{enter1,enter2,exit1,exit2}.
```

- In questo modo possiamo comporre l'LTS risultante con un altro LTS che usa azioni di richiesta senza che avvenga sincronizzazione

Esempio: mutua esclusione (3)



Un multi-insieme è una collezione di elementi non ordinata in cui gli elementi possono ripetersi, a differenza di un insieme tradizionale dove ogni elemento è unico; la ripetizione è significativa e definita dalla molteplicità, che indica quante volte un elemento appare, rappresentata formalmente da una coppia (A, m) , dove (A) è l'insieme degli elementi e (m) è la funzione che assegna ad ogni elemento il suo conteggio (molteplicità).

Dopo aver analizzato le Process Algebra (come FSP) che sono un approccio testuale ed algebrico per modellare i sistemi concorrenti, è naturale passare alle Reti di Petri, che offrono un approccio grafico e visivo.

Reti di Petri (Petri nets) →

Una Rete di Petri è un grafo composto da due tipi di nodi:

- Piazze (Places): Sono rappresentate graficamente da cerchi. Rappresentano le condizioni o gli stati parziali del sistema.
- Transizioni (Transitions): Sono rappresentate da rettangoli. Rappresentano gli eventi o le azioni che possono modificare lo stato del sistema.
- Archi: Connettono Piazze a Transizioni o Transizioni a Piazze (mai diretti Piazze-Piazze o Transizioni-Transizioni).

■ Approccio “grafico” alla rappresentazione dei sistemi concorrenti

- Può essere considerato un’**estensione naturale degli automi** per sistemi concorrenti
- Gli stati sono detti “**piazze**” (place)
- Più piazze sono contemporaneamente **attive**
 - indicato dalla presenza di “**token**” al loro interno
- Le **transizioni** modificano gli stati attivi:
 - **Consumano** un multi-insieme di stati attivi (token)
 - **Generano** un nuovo multi-insieme di stati attivi (token)

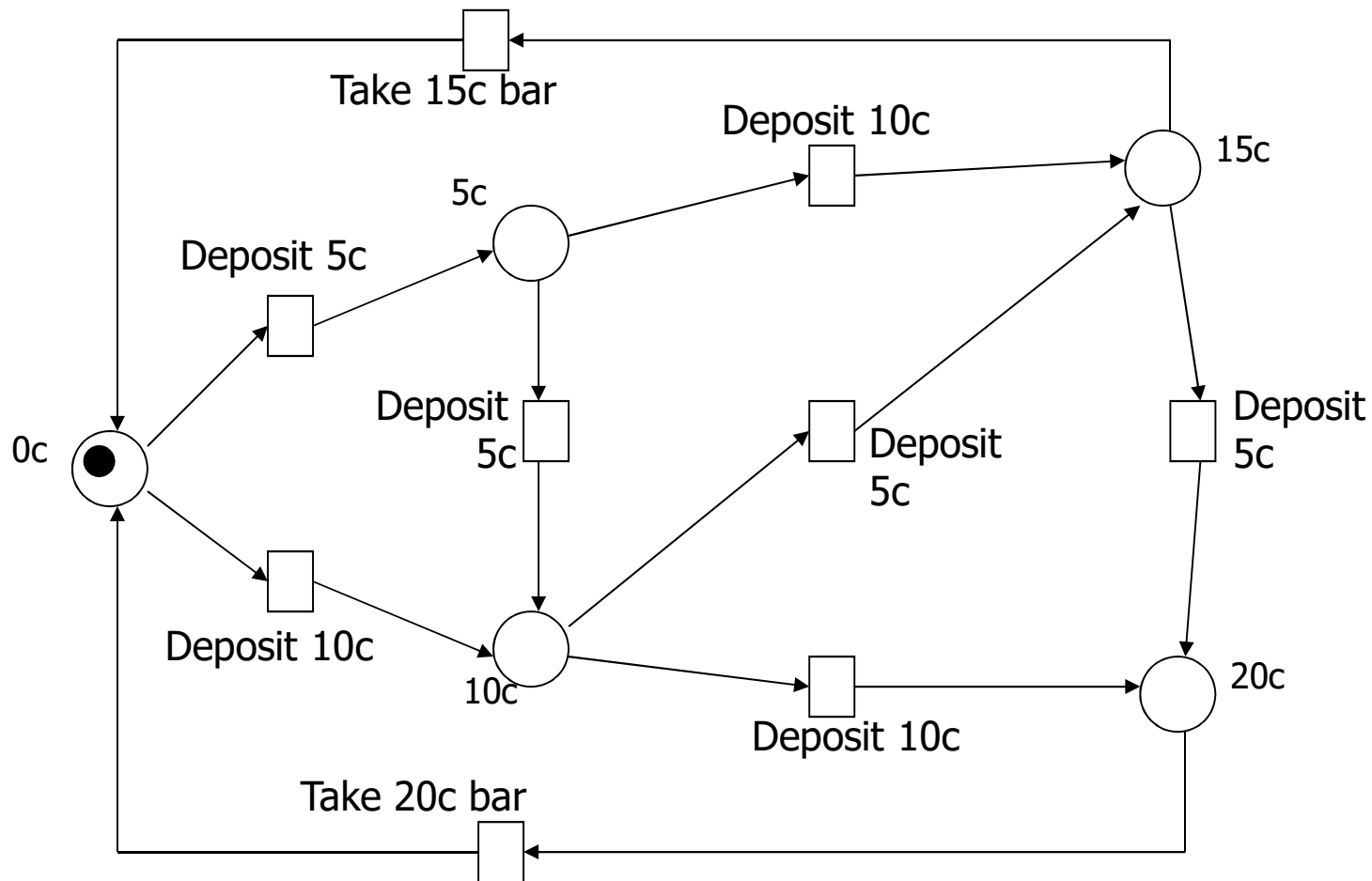
Le transizioni sono gli elementi dinamici che cambiano il Marking (stato) del sistema, simulando l'esecuzione di un'azione.

- Abilitazione (Enabling): Una transizione t è abilitata a scattare se tutte le piazze di input (quelle che la precedono) contengono un numero sufficiente di token (secondo la molteplicità degli archi di input).
- Scatto (Firing): Quando una transizione abilitata scatta:
 - > Consuma un multi-insieme di stati attivi (token) dalle piazze di input.
 - > Genera un nuovo multi-insieme di stati attivi (token) nelle piazze di output (quelle che la seguono).

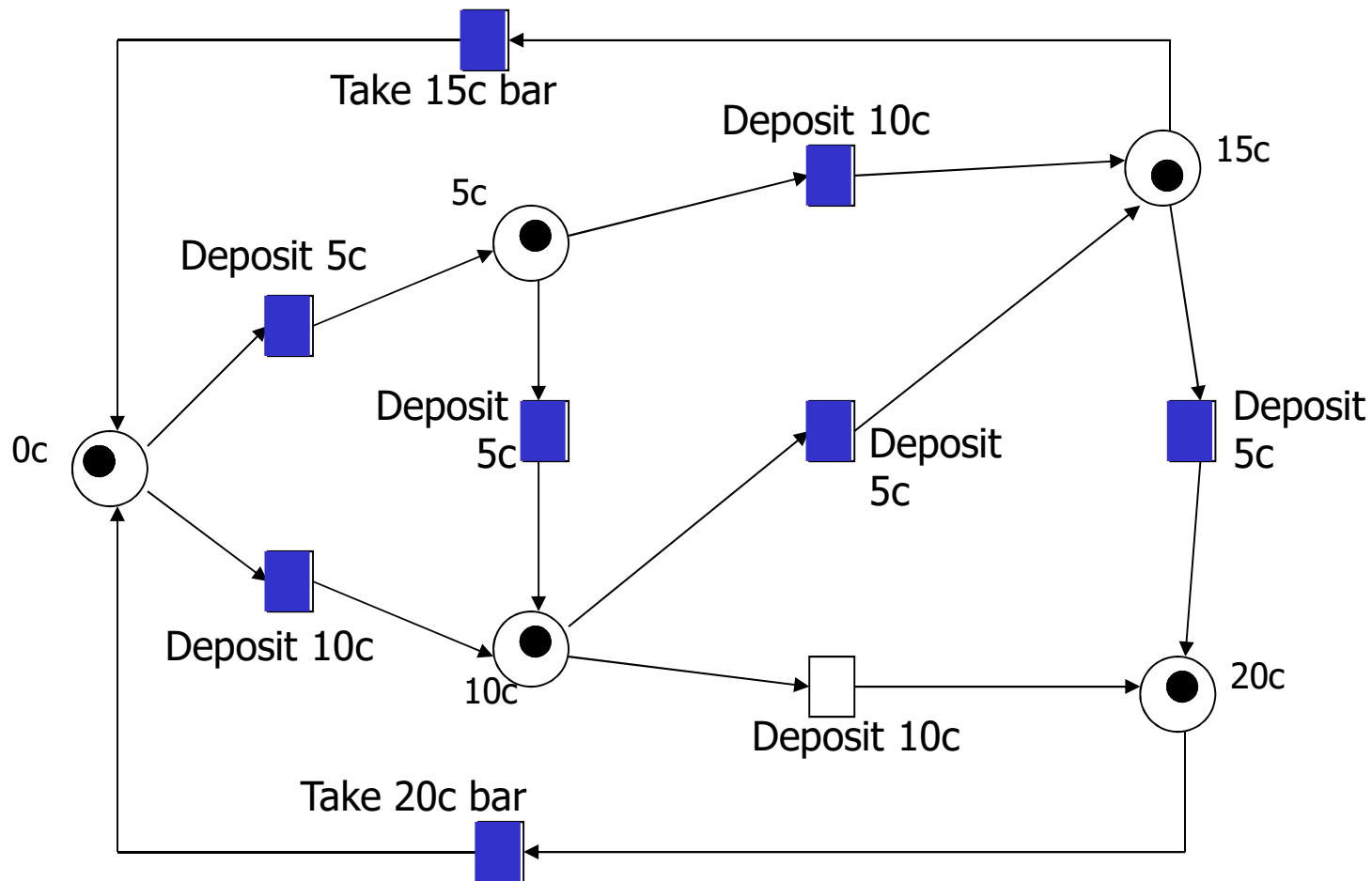
Esempio: distributore automatico

- Il distributore fornisce due tipi di snack,
da 15 e 20 centesimi
- Si usano monete da 10 e 5 centesimi
- Il distributore non prevede resto

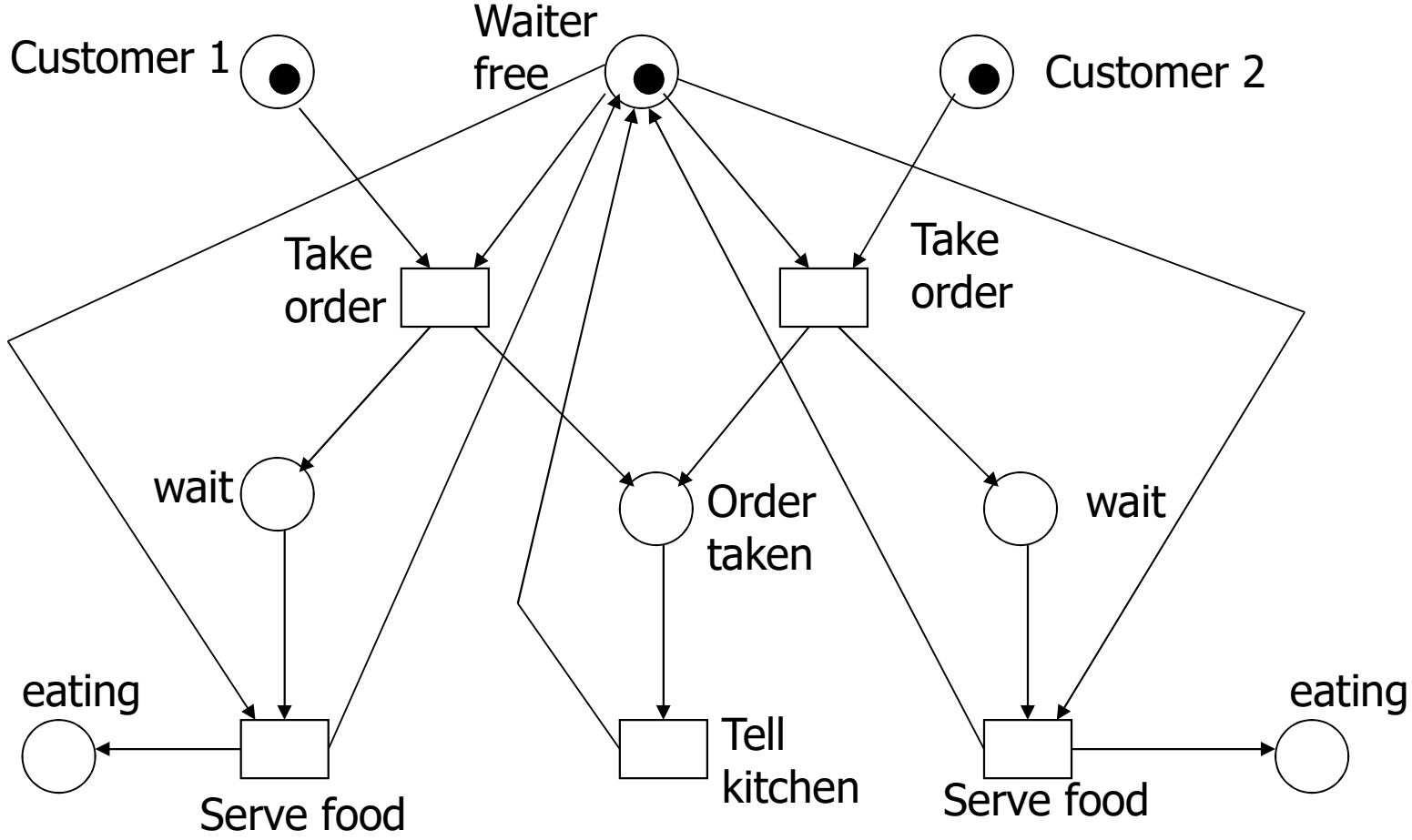
Esempio: distributore automatico



Esempio: distributore automatico

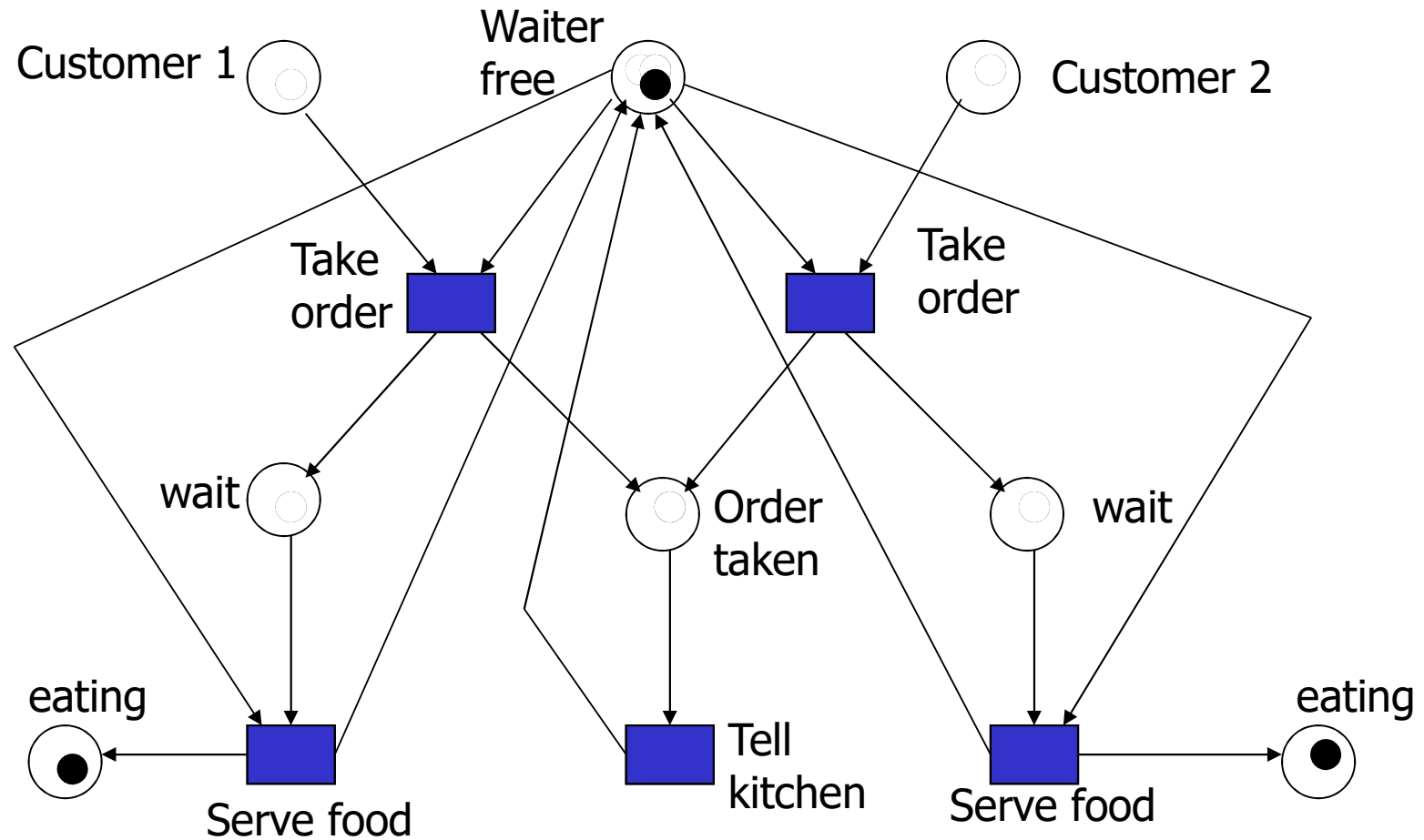


Esempio: ristorante

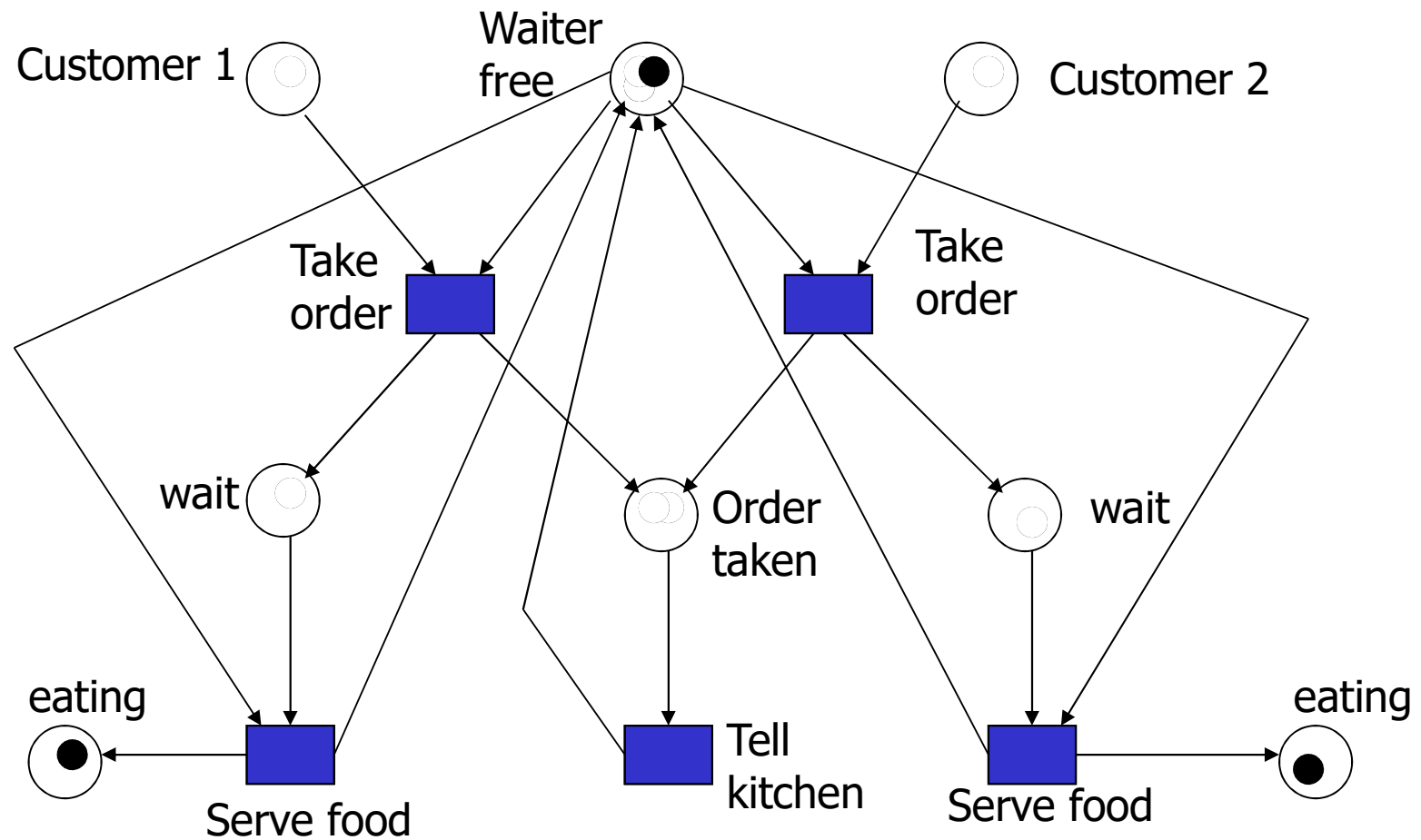


Una transizione viene eseguita se gli archi in entrata alla transizione hanno le rispettive piazze con token.
Inoltre, il numero di token prodotti/consumati dipende dal numero di archi in uscita/entrata

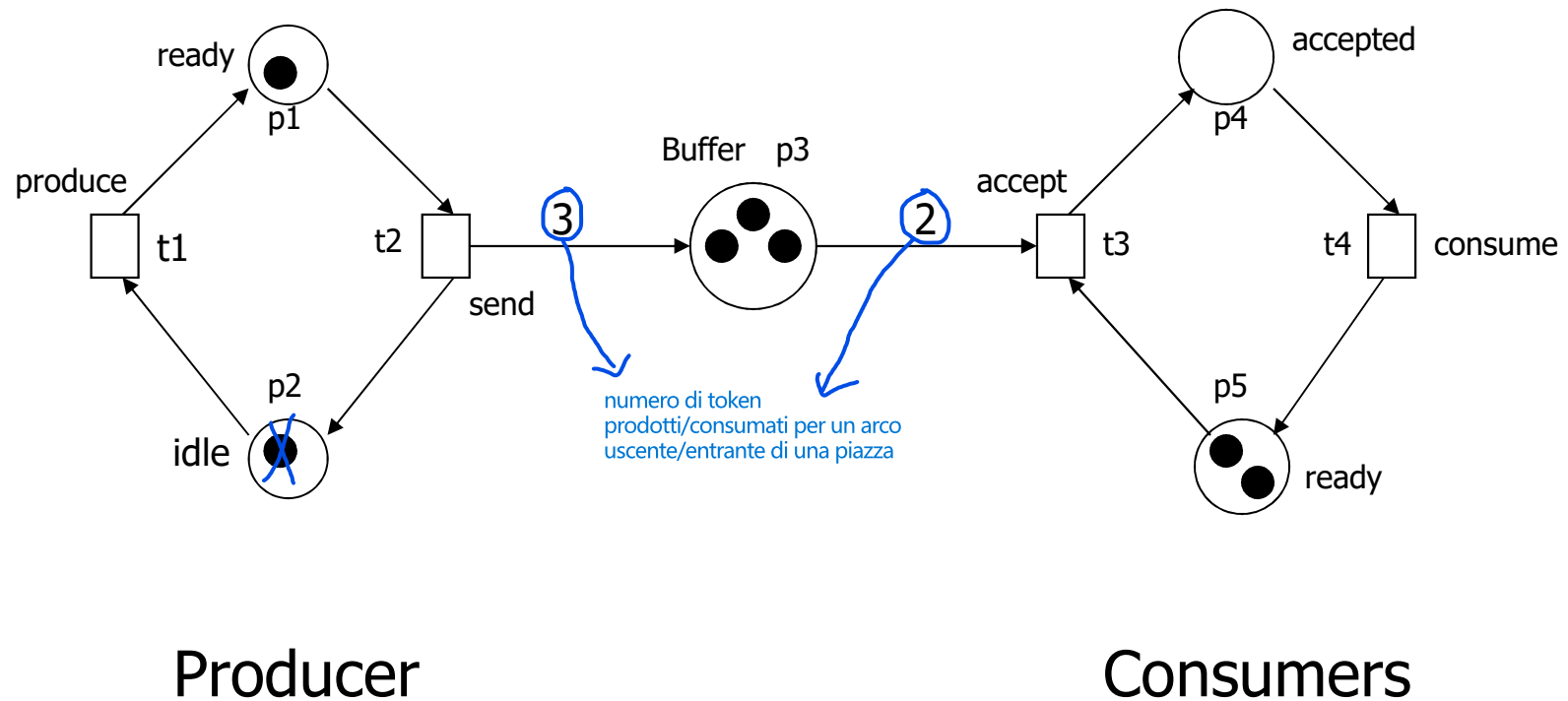
Esempio: ristorante (scenario 1)



Esempio: ristorante (scenario 2)

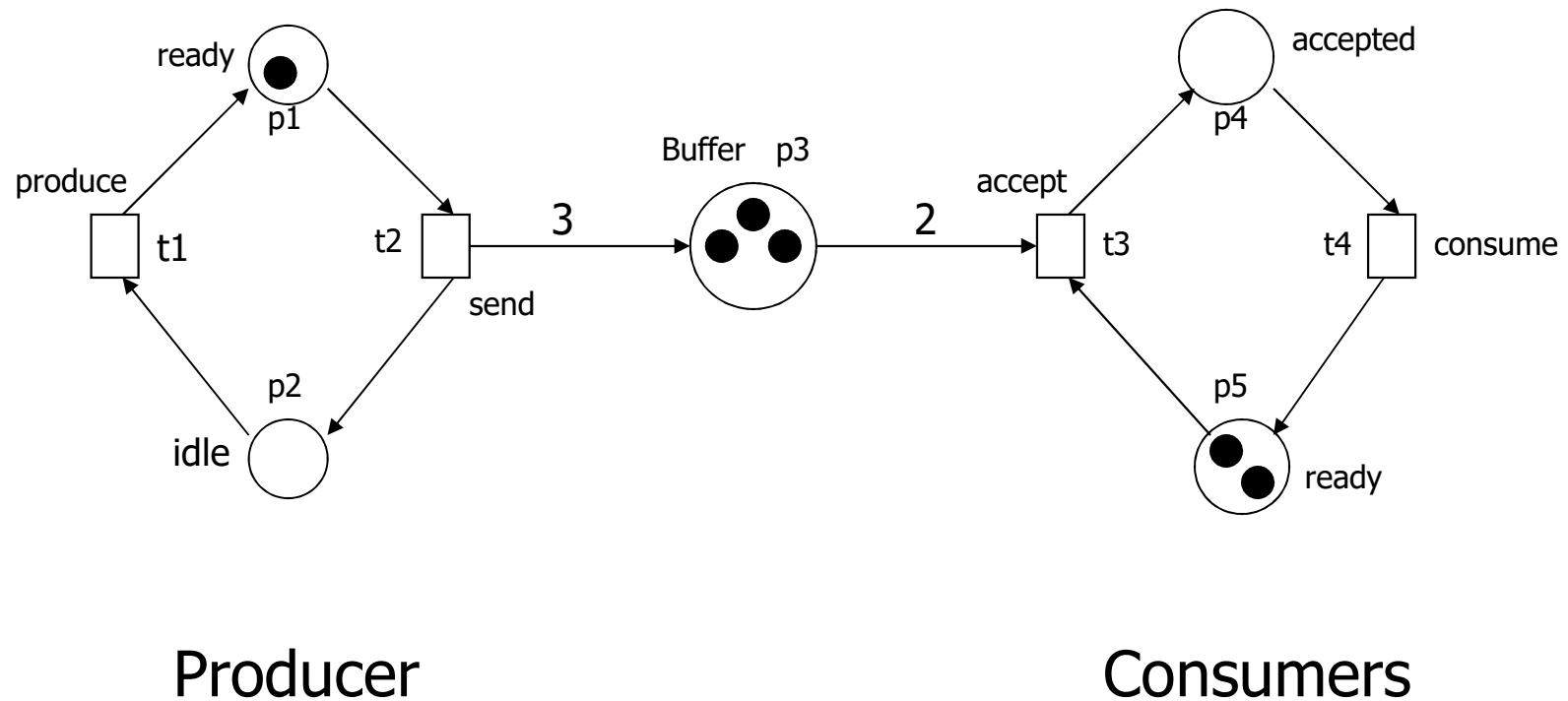


Esempio: produttore / consumatori

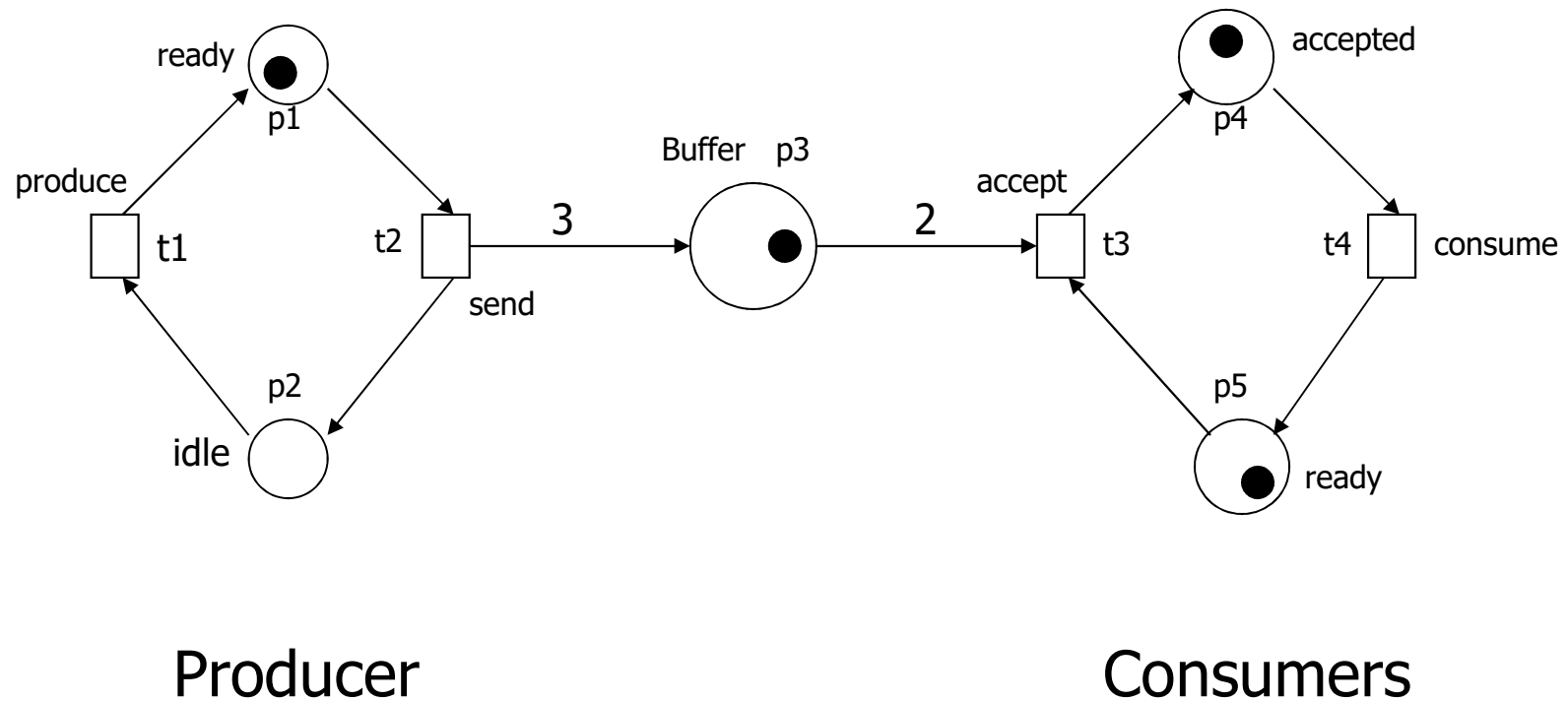


Le reti di petri sono non deterministiche: ho la possibilità di eseguire una delle tante transizioni disponibili (esempio: posso eseguire transizione t2 e p3 perché ho sia il token disponibile in 'ready' sia i token disponibili in 'Buffer' che sono maggiori di 2)

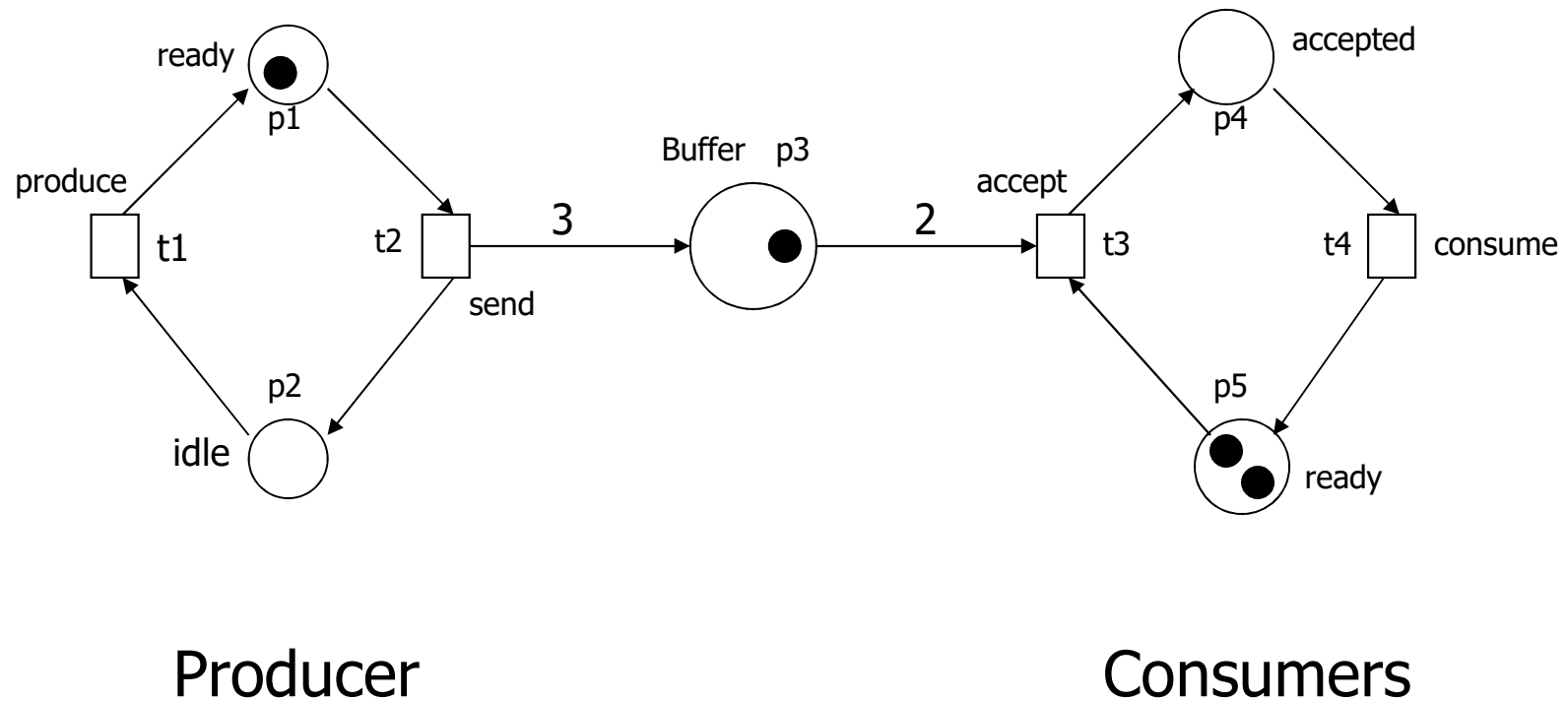
Esempio: produttore / consumatori



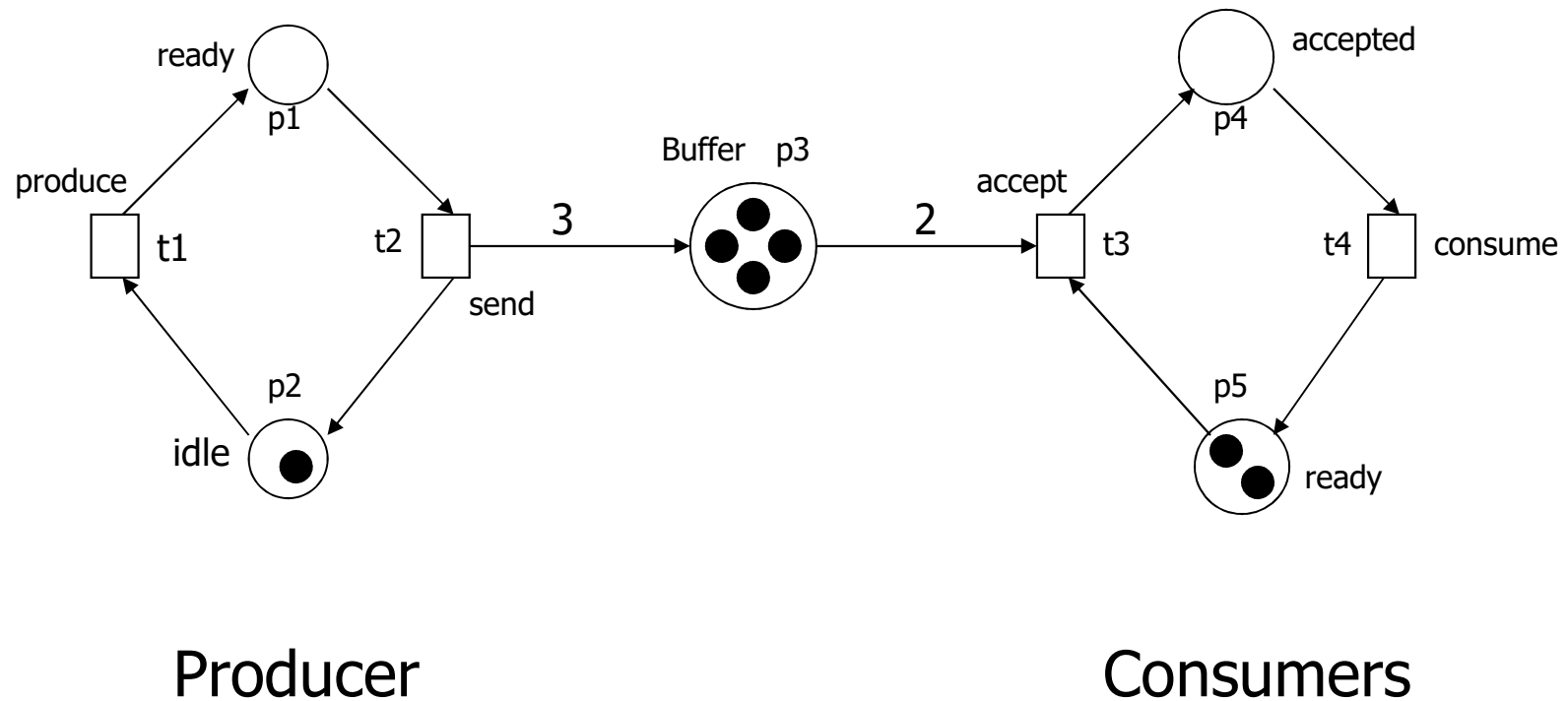
Esempio: produttore / consumatori



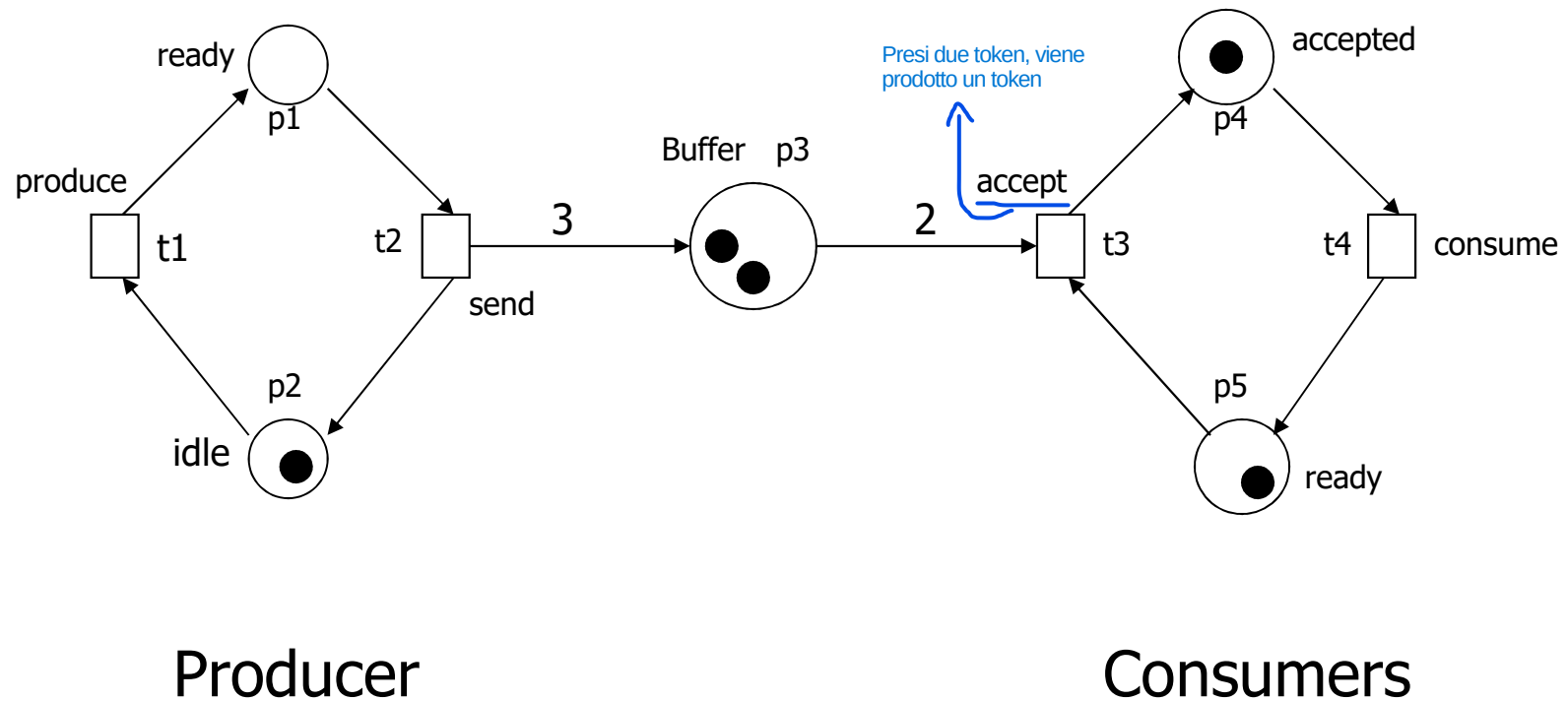
Esempio: produttore / consumatori



Esempio: produttore / consumatori



Esempio: produttore / consumatori



Gli archi senza numero hanno implicitamente peso 1

M_0 (Initial Marking - Marcatura Iniziale): M_0 è la marcatura iniziale. La marcatura M (lo stato attuale del sistema) è una funzione che assegna un numero intero non negativo di token a ciascuna piazza p in P . M_0 definisce la distribuzione dei token all'inizio dell'esecuzione del sistema. La dinamica del sistema è governata dalla Marcatura M e dalla regola di scatto delle transizioni. Lo scatto di una transizione trasforma la marcatura corrente M_i in una nuova marcatura M_{i+1} .

Definizione formale di rete di Petri

- Una **Petri Net** (PN) è una quintupla del seguente tipo:

- **$PN = (P, T, F, W, M_0)$**

- **$P = \{p_0, p_1, \dots, p_m\}$: a finite set of places** è un insieme finito e non vuoto di piazze. Le piazze rappresentano le condizioni o gli stati parziali del sistema.

- **$T = \{t_1, t_2, \dots, t_n\}$: a finite set of transitions** è un insieme finito e non vuoto di transizioni. Le transizioni rappresentano gli eventi o le azioni del sistema.

Esempio formale: Se vedi scritto $F = \{(p_1, t_1), (t_1, p_2)\}$, sai immediatamente che la transizione t_1 prende un token da p_1 e lo sposta in p_2 .

- **$F \subseteq (P \times T) \cup (T \times P)$: a set of arcs (flow relation)** è un insieme di archi orientati. Definisce la connettività della rete: un arco può andare solo da una Piazza a una Transizione o da una Transizione a una Piazza (è un grafo bipartito, cioè non va da una piazza ad una piazza, nemmeno da una transizione ad una transizione).

- **$W: F \rightarrow \{1, 2, 3, \dots\}$ weight function**

- **$M_0: P \rightarrow \{0, 1, 2, \dots\}$ initial marking**

Quanti token disponibili in ogni piazza

- **$P \cap T = \emptyset$**

(L'intersezione è l'insieme vuoto): Piazze e Transizioni sono insiemi di nodi disgiunti (una piazza non è una transizione, e viceversa).

- **$P \cup T \neq \emptyset$**

(L'unione non è l'insieme vuoto): La rete non è vuota.

è la funzione di peso. Assegna un peso (un intero positivo) a ciascun arco nella relazione di flusso F . Questo peso determina quanti token devono essere consumati da una piazza di input o generati in una piazza di output.

Stati (Nodi) del Marking Graph
- Configurazione della PN (Marking): Gli stati del Marking Graph sono i Marking (M) della Rete di Petri.
> Un marking è un multi-insieme di piazze che indica la quantità di token presenti in ciascuna piazza in un dato momento.
- Configurazione Iniziale (q_0): Lo stato iniziale dell'LTS è la Marcatura Iniziale (M_0) della PN.

Il Marking Graph (Grafo di Marcatura) è il modo in cui una Rete di Petri viene formalmente trasformata in un LTS (Labeled Transition System), permettendo così l'analisi del suo comportamento dinamico, compresa la verifica con LTL.

Transizioni (Archi) del Marking Graph
- Passaggio di Configurazione: Le transizioni del Marking Graph sono etichettate dalle Transizioni (T) della Rete di Petri.
- Funzione di Transizione (delta): Un arco (una transizione) esiste tra due stati M_i e M_j se e solo se l'esecuzione di una transizione t in T della Rete di Petri fa passare il Marking da M_i a M_j .
> L'etichetta dell'arco è l'azione t della PN.
> Questo passaggio di configurazione indica in che modo l'esecuzione di t modifica il marking corrente (consumando token di input e generando token di output).

Marking graph

- Il marking graph di una Petri Net è un **LTS** che ne definisce il comportamento
 - **Configurazione** della PN (chiamato **marking**): un **multi-insieme di piazze** che indica la quantità di token presenti in ciascuna piazza
 - **Passaggio di configurazione**: indica in che modo l'**esecuzione di una transizione** della rete di Petri modifica il marking corrente
 - **Configurazione iniziale**: marking iniziale M_0

Utilizzo per la Verifica

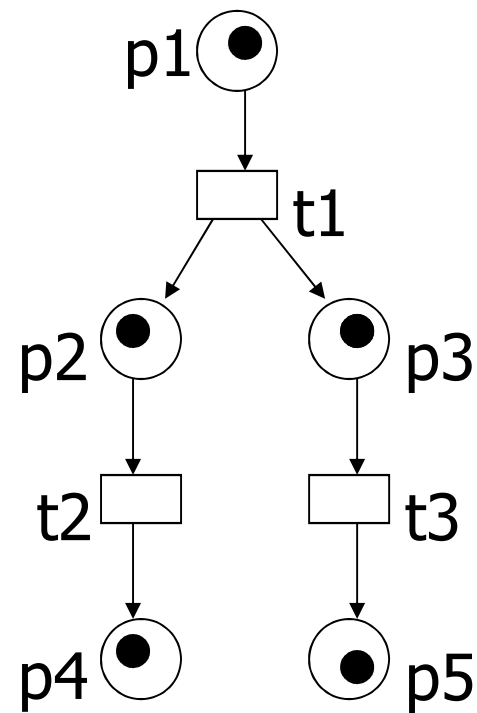
Una volta generato il Marking Graph (LTS), è possibile applicare tutti gli strumenti di Model Checking e le logiche temporali LTL:

- Safety: Verificare se un marking indesiderato (es. un marking che indica un deadlock o un errore) è irraggiungibile.
- Boundedness: Verificare se il numero di token in qualsiasi piazza non supera mai un certo limite (essenziale per la finiteness dell'LTS).

In sintesi, il Marking Graph è l'LTS che cattura tutti gli stati raggiungibili e tutte le sequenze di azioni possibili di una Rete di Petri.

Definendo le transizioni da un Marking ad un altro, ho una traccia massimale specifica LTS

Esempio: markings



$$M0 = (1, 0, 0, 0, 0)$$

$$M1 = (0, 1, 1, 0, 0)$$

$$M2 = (0, 0, 1, 1, 0)$$

$$M3 = (0, 0, 0, 1, 1)$$

Si vede anche qua che le reti di Petri sono non-deterministiche perché qua potevo applicare sia t2 sia t3 in sequenze diverse (cioè potevo applicare o prima t2 o prima t3 seguita dall'altra transizione)

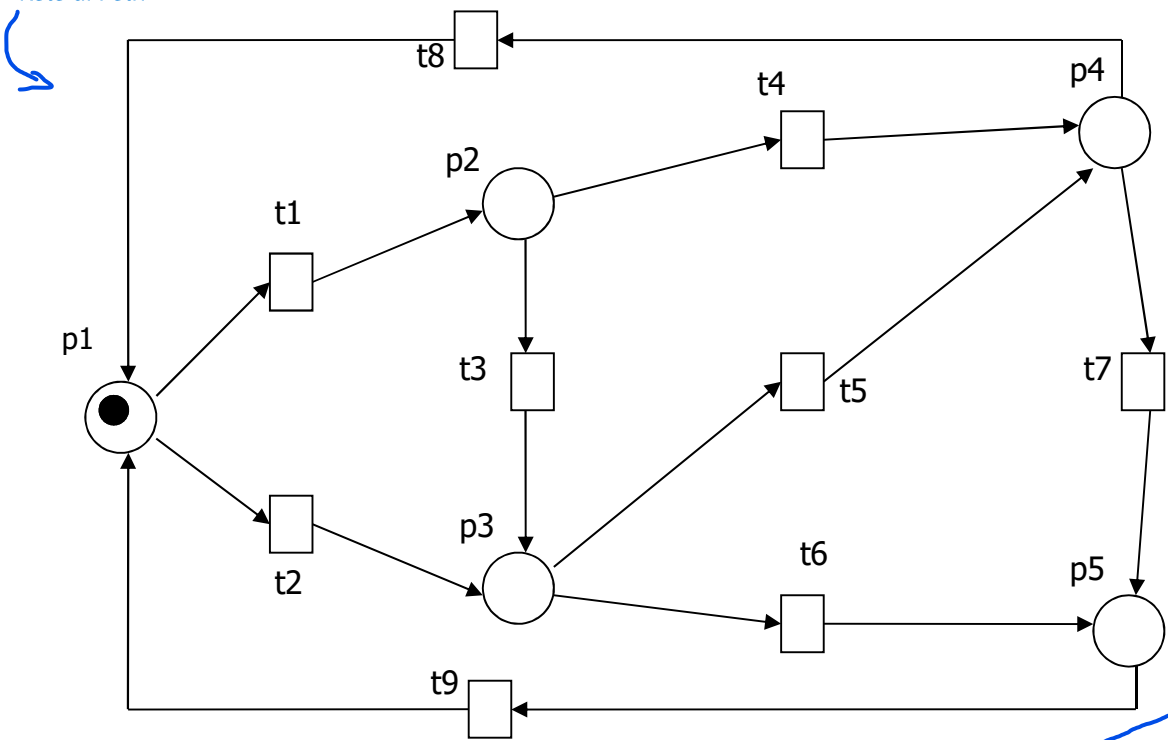
Il Marking Graph è una rappresentazione del comportamento della Rete di Petri, e questa rappresentazione coincide con la definizione di un LTS: è un modello matematico astratto usato per descrivere il comportamento di un sistema in termini di stati e transizioni etichettate tra questi stati.

Modellare un sistema con una Rete di Petri e poi analizzare il suo Marking Graph permette di usare gli strumenti teorici ed algoritmici sviluppati per gli LTS (come il model checking) per verificarne le proprietà

Marking: Indica lo stato attuale del sistema, ovvero quanti "token" sono presenti in ciascun posto.

Esempio: marking graph

Rete di Petri

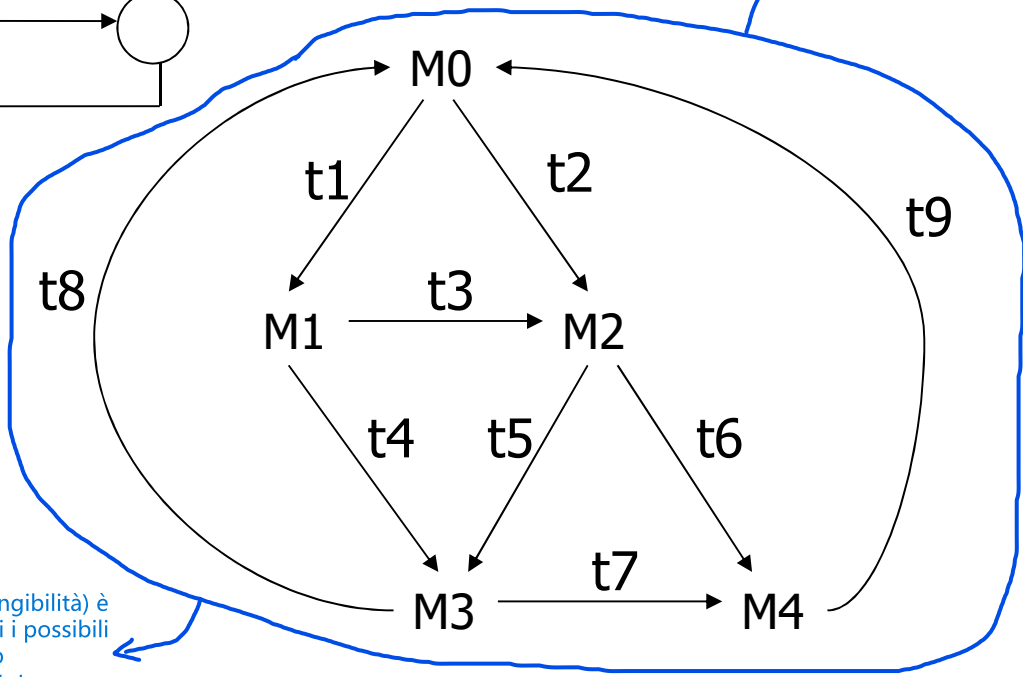


Marking iniziale: M_0

- $M_0 = (1,0,0,0,0)$
- $M_1 = (0,1,0,0,0)$
- $M_2 = (0,0,1,0,0)$
- $M_3 = (0,0,0,1,0)$
- $M_4 = (0,0,0,0,1)$

Lista di Markings della Rete di petri che diventano gli stati del Marking Graph

Un Marking Graph è considerato un LTS



Se invece di definire una traccia specifica, definisco tutte le tracce possibili, ottengo un LTS dove le transizioni della reti di petri sono le transizioni nel LTS, mentre i markings della rete di petri sono gli stati del LTS

Un Marking Graph (o Grafo di Raggiungibilità) è una rappresentazione che mostra tutti i possibili stati (marking) che la Rete di Petri può raggiungere e come passare da uno stato all'altro attraverso l'attivazione delle transizioni.

$W(p,t)$ è il peso dell'arco che va dalla piazza p alla transizione t (cioè l'input di t). Se l'arco non esiste, il peso $W(p,t)$ è assunto come 0.

Definizione formale completa di come una Rete di Petri evolve il suo stato (Marking) attraverso lo scatto di una transizione. Questo definisce la dinamica del sistema e la funzione di transizione (delta) del suo Marking Graph (LTS).

- Se p è solo una piazza di input per t (cioè $W(p,t) > 0$ e $W(t,p) = 0$), $M'(p)$ diminuisce.
 - Se p è solo una piazza di output per t (cioè $W(p,t) = 0$ e $W(t,p) > 0$), $M'(p)$ aumenta.
 - Se p è una piazza di ciclo (cioè $W(p,t) = W(t,p) > 0$), $M'(p)$ rimane invariato.
 - Se p non è connessa a t , $M'(p) = M(p)$ (il marking non cambia).

Formalmente

- Un **marking** M è un multi-insieme di piazze:

$$M : \overbrace{P}^{\text{insieme di piazze}} \rightarrow \overbrace{\{0, 1, 2, \dots\}}^{\text{insieme dei numeri naturali}}$$

- Una **transizione** $t \in T$ di una $\overbrace{PN}^{\text{Rete di Petri}}$ è **abilitata** in un marking M se:

$$\text{Per ogni } p \in P \quad M(p) \geq \overbrace{W(p, t)}^{\text{Peso dell'arco in input alla transizione dalla piazza } p}$$

In altre parole, La transizione t può scattare solo se tutte le sue piazze di input contengono almeno il numero di token richiesto dal peso del rispettivo arco.

Una transizione t in T è abilitata (può scattare) in un marking M se, per ogni piazza p in P , la quantità di token in p è sufficiente a coprire i requisiti di input di t .

- L'**esecuzione** di una $t \in T$ abilitata nel marking M fa **passare** la PN al marking M' dato da:

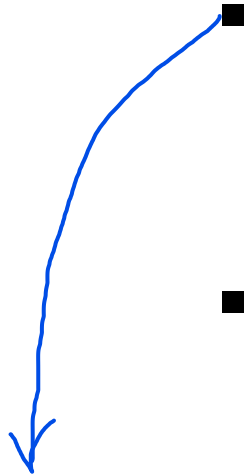
$$\text{Per ogni } \overbrace{p \in P}^{\text{piazza}} \quad M'(p) = \overbrace{M(p)}^{\text{Il numero di token iniziali nella piazza } p} - \overbrace{W(p, t)}^{\text{La quantità di token consumati da } p \text{ (l'input di } t\text{)}} + \overbrace{W(t, p)}^{\text{La quantità di token generati in } p \text{ (l'output di } t\text{)}}.$$

Nota: si assume che il peso W di un arco non in F sia 0

La proprietà che determina se il Marking Graph (LTS) è finito o infinito è la Boundedness della Rete di Petri.

Marking graph (continua)

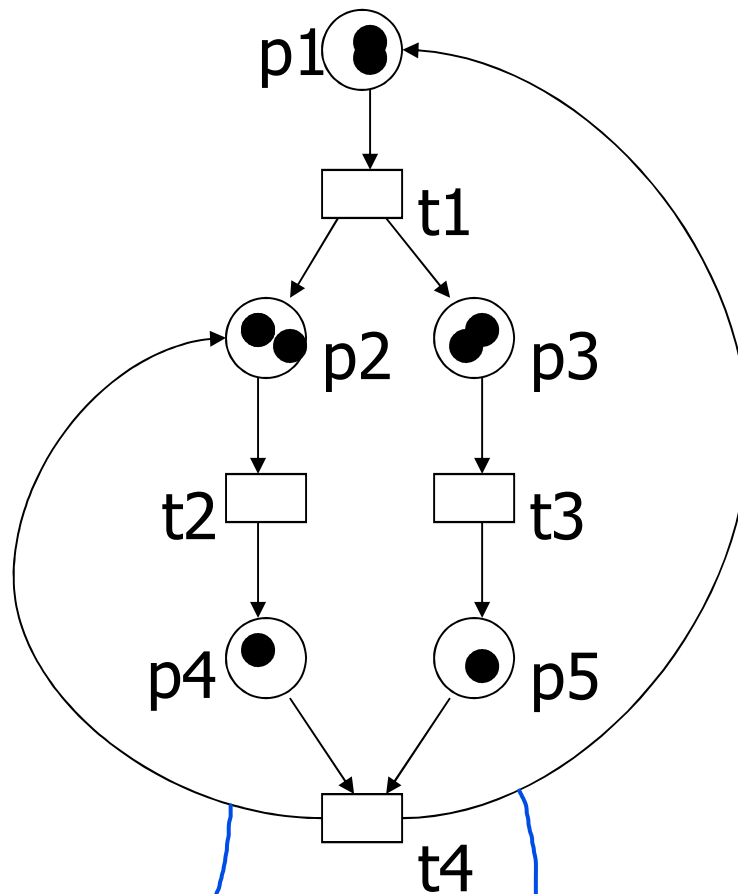
- A volte il marking graph risulta essere **infinito**:
 - Succede quando la rete di Petri è “unbounded”, cioè può produrre una quantità illimitata di token
 - Esempio: si veda prossima slide



Una Rete di Petri è definita unbounded (illimitata) quando può produrre una quantità illimitata di token in una o più piazze.
- Effetto sul Marking Graph: Se una piazza p_i può accumulare $0, 1, 2, 3, \dots$ token, l'insieme dei marking M può contenere un numero infinito di marking possibili.
- Conseguenza: Il Marking Graph risulta essere infinito.

Implicazione per il Model Checking
- L'infinità del Marking Graph ha gravi conseguenze per la verifica formale:
- Stato Esplosione: Se la rete è unbounded, il Model Checker non può terminare l'esplorazione dell'LTS, rendendo la verifica di alcune proprietà indecidibile.
- Decidibilità: Per poter applicare algoritmi di Model Checking LTL (che richiedono un numero finito di stati), è necessario che la Rete di Petri sia bounded (limitata da un numero finito di token per piazza).

Marking graph (continua)



Rimuovendo uno di questi archi,
renderei la Petri Net Bounded

$$M0 = (1, 0, 0, 0, 0)$$

$$M1 = (0, 1, 1, 0, 0)$$

$$M3 = (0, 0, 0, 1, 1)$$

$$M4 = (1, 1, 0, 0, 0)$$

$$M5 = (0, 2, 1, 0, 0)$$

Rete di Petri unbounded

Perché ho un numero infinito di Marking della Rete di Petri ed un numero infinito di token per piazza, dato che in questo caso continua ad aumentare linearmente il numero di token che passa nella rete di petri

Se la PN è unbounded (illimitata), il suo Marking Graph (LTS) è infinito, rendendo l'esplorazione completa impossibile e la verifica indecidibile.

Il Model Checking basato su tecniche di esplorazione dello spazio degli stati è applicabile solo alle Reti di Petri bounded

Una volta generato l'LTS, si verifica se una formula di LTL (ϕ) è soddisfatta:
- Valutazione sulle Tracce: Un LTS soddisfa una formula LTL se i modelli (π) individuati da tutte le sue tracce massimali sono tali che $\pi \models \phi$.
- Traccia Massimale: Una sequenza infinita di stati (Marcature).

Model checking

- Su Petri nets "bounded" si verifica se una **formula** di LTL è **soddisfatta** come già visto:

- Si considera il **marking graph** come un **LTS** che rappresenta il comportamento della rete di Petri
- E poi i **modelli** π di tutte le sue tracce massimali

- Ora però **proposizioni** da **stati** dell'LTS:

- In questo caso gli stati sono i **marking** della PN
- Sono definite proposizioni che consentono di osservare le piazze: **proposizione "p"** vera se e solo se **piazza p contiene almeno un token**

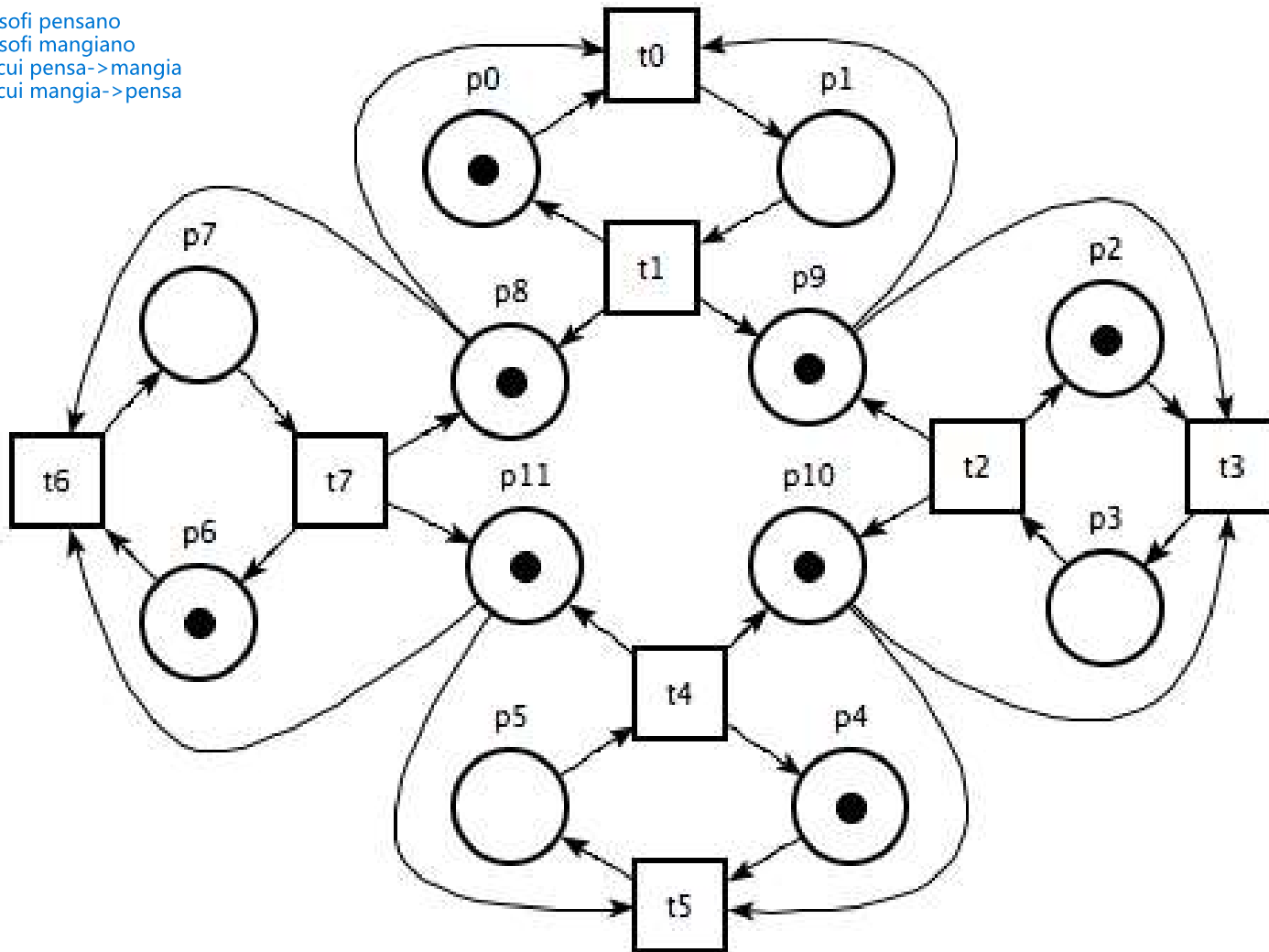
In LTL, le formule sono composte da proposizioni atomiche p . In questo contesto, le proposizioni sono definite sulle Marcature:
Una proposizione " p " (dove p è il nome della piazza) è vera se e solo se la piazza p contiene almeno un token.
Formalmente: Dato uno stato (Marcatura M) del Marking Graph, la proposizione p è vera se:
 $M \models p$ se e solo se $M(p) \geq 1$

In uno stato (marking) del LTS,

Un filosofo per mangiare, deve avere le due
forchette (che si trovano nelle piazze p8,p9,p10,p11)

Esempio: 4 dining philosopher

p0,p2,p4,p6 sono le piazze dove i filosofi pensano
p1,p3,p5,p7 sono le piazze dove i filosofi mangiano
t0,t3,t5,t6 sono le transizioni tramite cui pensa->mangia
t1,t2,t4,t7 sono le transizioni tramite cui mangia->pensa



Alcune proprietà esprimibili in LTL (alcune vere, alcune false – usare TINA per controllare):

- FALSE (traccia: eseguo t0 e t5 quando ho le forchette a disposizione di entrambi i filosofi lontani)

